MS&E 244: Homework 2

January 27, 2026

Your Name(s) Here Björn Thor Stefánsson, Sondre Rodge, Kwun Yu (Kaden) Wu

Introduction

Welcome to MS&E 244: Statistical Arbitrage - Homework 2.

This assignment is due by January 27th, 2026 at 11:59pm Pacific Time and should be done in groups of 2-3 people. We recommend collaborating via a private GitHub repo.

Instructions for this assignment are below. **Please read all instructions carefully.**

1. Replace "Your Name(s) Here" above in bold with your name, and, if working in a group, your group members' names.
2. Read and run the instructions code in the Setup section. Make sure you understand what it does.
3. Answer the questions in the sections below. Make sure to limit code output to a reasonable length so that the resulting PDF you'll make from this notebook is readable. If the resulting PDF is not readable, keep your lines of code less than 80 characters.
4. Once finished, export or convert the notebook to PDF, using any method you like. For example, you can use the File -> Save and Export Notebook As -> PDF option in Jupyter Lab, you can use the nbconvert command line tool, etc. Whatever displays best is fine. Make sure no code is cut off in the PDF and that sheets are 8.5x11 inch dimensions or similar.
5. Submit the resulting PDF to the Gradescope assignment. If working in a group, only one person needs to submit and select all group members on Gradescope.
6. Zip and submit all files (your .ipynb file and any .py files we provided or you created) to the Canvas assignment. If working in a group, only one member of the group needs to submit to Canvas. Unlike Gradescope, no group members need to be selected on Canvas.

Setup

System Requirements

First, ensure you have the proper system requirements:

1. We will only support Mac or Linux in this course. If you're on Windows, learn to use WSL 2.0 to run Linux on Windows, and then run these commands via WSL.
2. Ensure you have Python 3.10 and pip installed. You also need something to run this Jupyter notebook, whether that's Jupyter Notebook, Jupyter Lab, an IDE like VS Code, or Google Colab. We recommend collaborating via GitHub and using Jupyter Lab or VS Code to complete this notebook.

Jupyter Lab

To use Jupyter Lab, we'll need to set up the Python virtual environment and install the requirements. We've provided example instructions for doing so below for those using Jupyter Lab. Note that you may need to substitute `pip` with `pip3` depending on your installation. You can learn more by reading the venv docs and the ipykernel docs. See documentation for Jupyter Lab if needed.

```
# Navigate to the root directory containing your course files
cd /path/to/mse244

# Place the requirements.txt file in this directory if it isn't already there
# Put this .ipynb file in this directory or in a subdirectory, e.g. "hw1"

# Create a virtual environment
python3.10 -m venv .venv

# Activate the virtual environment
source .venv/bin/activate

# Install the required packages
pip install -r requirements.txt

# Add the virtual environment to Jupyter
python -m ipykernel install --user --name mse244 --display-name="MSE244"

# Start Jupyter Lab
jupyter lab

# Once in Jupyter, make sure to select the "MSE244" kernel
```

VS Code

If using VS Code, read the docs for Jupyter notebooks in VS Code and virtual environments in VS Code. VS Code can help you create and activate a virtual environment and Jupyter kernel for this notebook through its GUI (though you should also learn how to do these things via the command line). Make sure that you use Python 3.10 to create your virtual environment.

Once setup is complete and you're running within your environment, you can begin executing the cells below.

Imports

[ ]:
```
%load_ext autoreload
%autoreload 2

import warnings
from datetime import datetime
from typing import Any, Dict, Optional, Tuple, Union
```

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mpl_ticker
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from statsmodels.graphics.tsaplots import plot_acf
from arch.unitroot import ADF

# We add any local imports (Python files we've written) here
from backtest import (
    load_data,
    plot_asset_with_max_return,
    select_asset_universe,
    form_pairs,
    estimate_hedge_ratio,
    compute_signal,
    allocate_positions,
    run_backtest,
)


# Configure pandas display options
pd.set_option('display.max_columns', 20)
pd.set_option('display.max_rows', 40)
pd.set_option('display.precision', 4)
```

```
/Users/kadenwu/anaconda3/lib/python3.10/site-
packages/pandas/core/arrays/masked.py:61: UserWarning: Pandas requires version
'1.3.6' or newer of 'bottleneck' (version '1.3.5' currently installed).
  from pandas.core import (
```

Data

To start with, we'll load the supplied Japanese equity data, preprocess the data, and set up the necessary variables.

Please make sure to look up and learn about any lines you don't understand by reading the Pandas User Guide. This is essential knowledge for anyone working with financial data in Python.

[ ]:
```python
config = {
    'NIKKEI_CSV_PATH': 'N225.csv'
}

prices, returns, tickers, metadata = load_data(config, verbose=True)
```

```
Price df shape at load: (4235, 226)
```

```
Price df shape after slicing time axis: (2703, 226)
Price df shape after removing future asset columns: (2703, 222)

Prices head:

              1332.T     1605.T    1721.T     1801.T    1802.T    1803.T  \
Date
2007-01-04  514.1772  1640.3337  906.6639  1236.6694  523.9589  456.3690
2007-01-05  535.0902  1598.5864  909.3971  1226.7228  513.8568  450.3043
2007-01-09  524.2733  1614.2418  925.7949  1256.5621  520.5914  460.9175
2007-01-10  506.2446  1581.1916  913.4961  1246.6157  511.8364  466.2240
2007-01-11  504.0811  1589.8885  910.7634  1249.9312  511.8364  479.8697

               1808.T    1812.T     1925.T     1928.T  …     9503.T  \
Date                                                   …
2007-01-04  1560.3464  682.8918  1352.8394  1015.7311  …  2240.7510
2007-01-05  1542.5345  677.7186  1339.8625  1005.1937  …  2172.8496
2007-01-09  1538.9718  695.8253  1372.3043  1015.7311  …  2179.6396
2007-01-10  1535.4095  691.9454  1356.0836  1006.3643  …  2138.8989
2007-01-11  1567.4717  699.7054  1330.1299  1010.4626  …  2138.8989

               9531.T     9532.T     9602.T    9613.T     9735.T     9766.T  \
Date
2007-01-04  2253.5034  1583.7520  1837.3094  946.7017  4466.2446  2745.0986
2007-01-05  2222.2046  1548.7904  1841.5720  956.0132  4409.3506  2601.4155
2007-01-09  2194.3833  1611.7211  1858.6237  977.7407  4508.9165  2586.2905
2007-01-10  2159.6069  1566.2709  1841.5720  957.5651  4430.6855  2510.6685
2007-01-11  2145.6963  1569.7675  1854.3606  945.1494  4380.9028  2525.7932

               9843.T     9983.T     9984.T
Date
2007-01-04  2307.3474  3180.9983  1098.4468
2007-01-05  2232.3472  3203.0117  1120.9558
2007-01-09  2214.6997  3167.2395  1114.2031
2007-01-10  2342.6404  3101.1982  1118.7048
2007-01-11  2347.0520  3057.1704  1118.7048

[5 rows x 221 columns]


Metadata head:

Ticker      Industry                Sector                Company
1332.T       Fishery        Consumer Goods           NISSUI.CORP.
1605.T        Mining             Materials            INPEX.CORP.
1721.T  Construction  Capital Goods/Others  COMSYS.HOLDINGS.CORP.
1801.T  Construction  Capital Goods/Others           TAISEI.CORP.
1802.T  Construction  Capital Goods/Others         OBAYASHI.CORP.
```

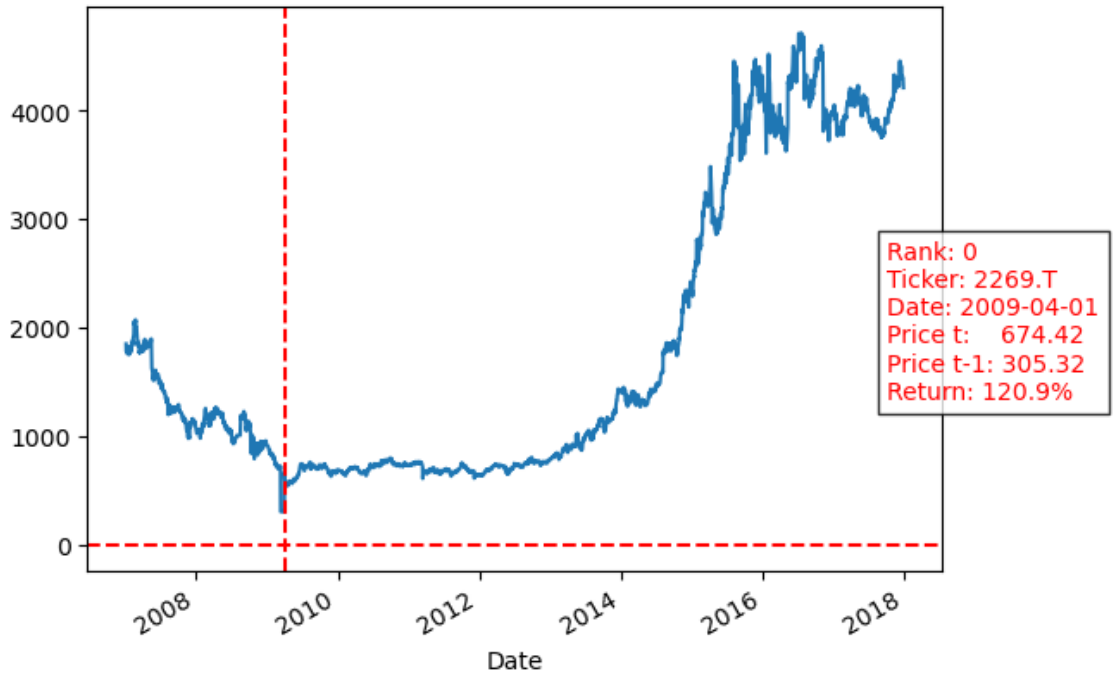
1 Question 1

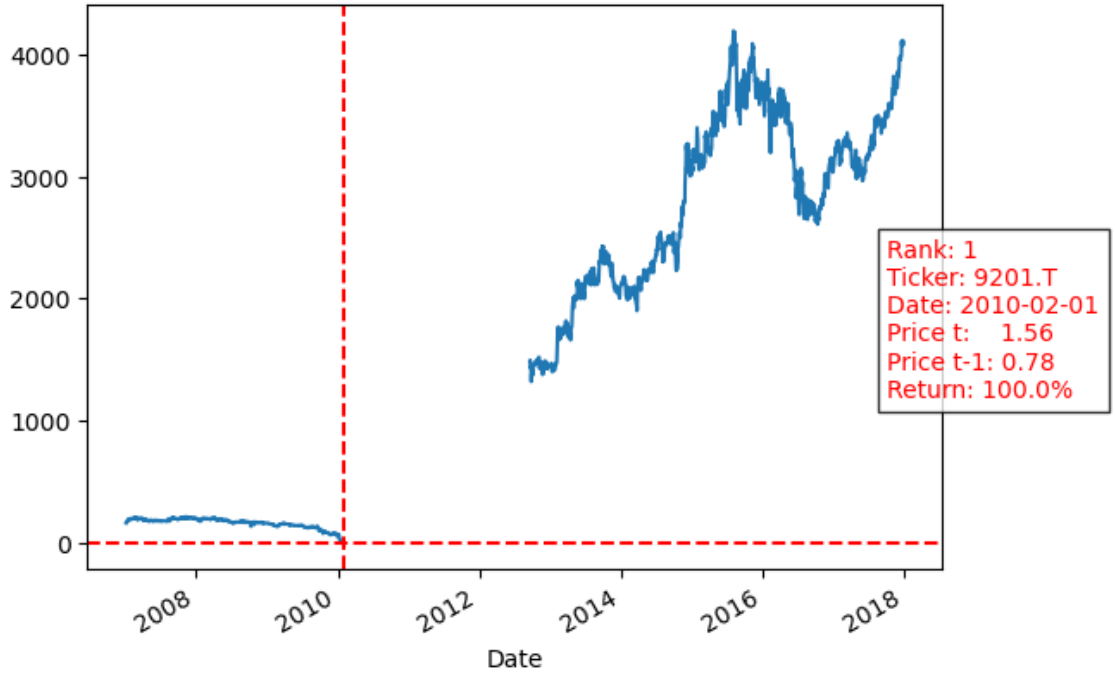
(2 points)

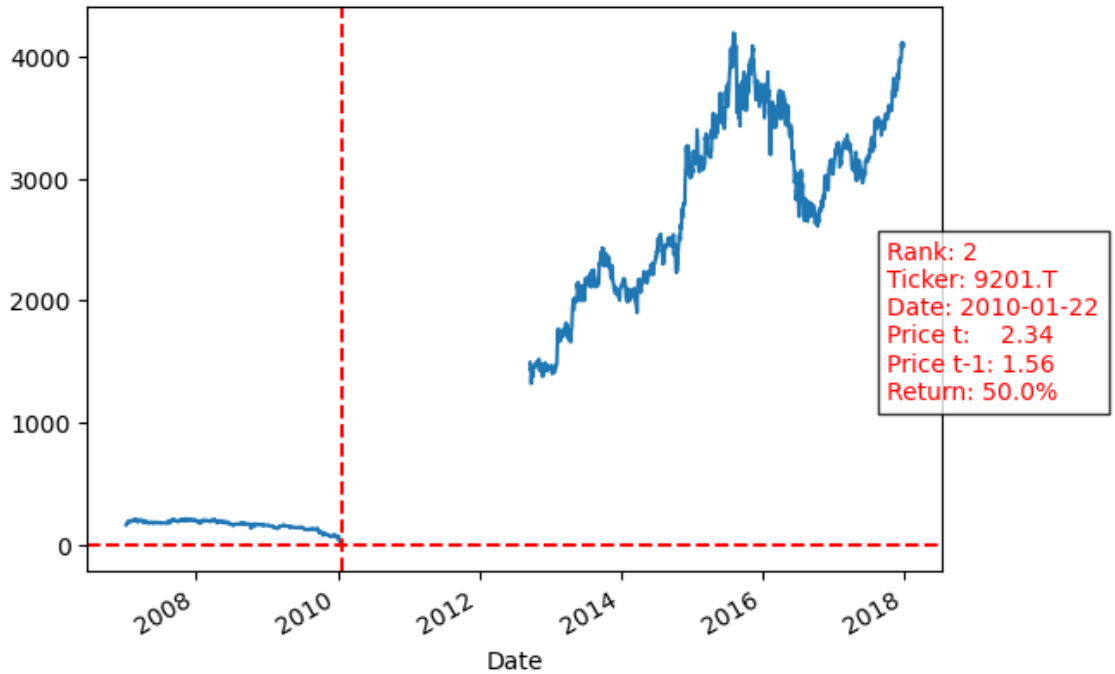
First, let's look into some issues with the data. Some of you may have noticed in the last homework that there were a few strange returns or prices. Although we won't grade based on this, these were intentionally left in to sharpen your data cleaning skills. If you noticed them, congrats! Now we will explore these issues and learn about what to do with them.

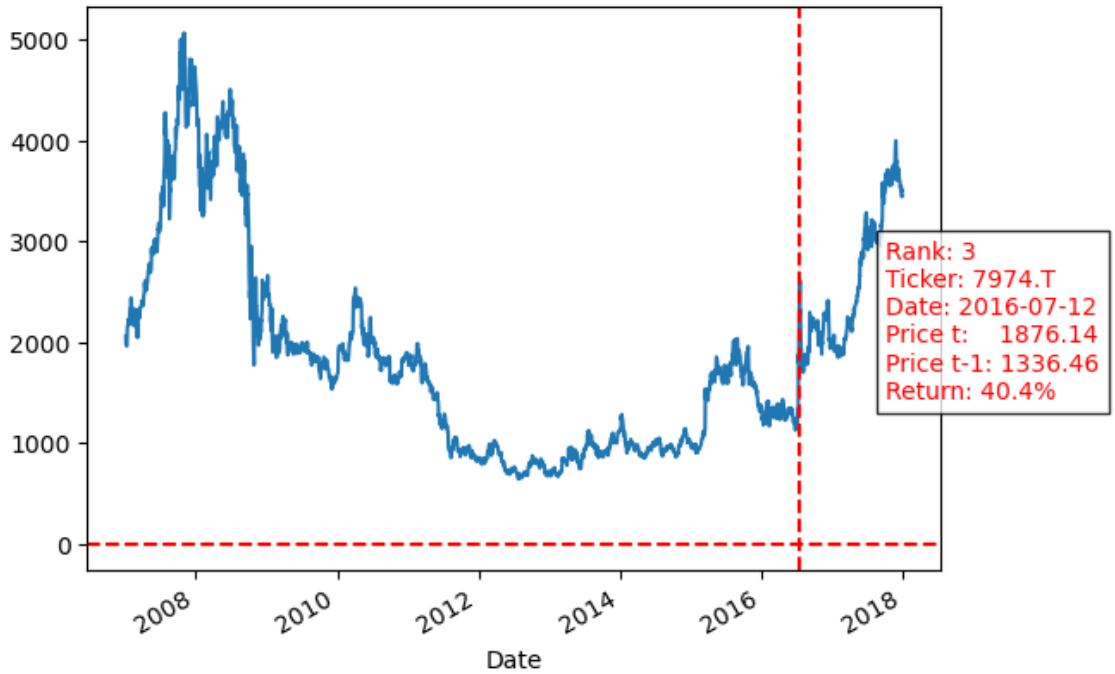
Below, we've written code to do just that. Take a look at the function called `plot_asset_with_max_return`, which is specified in the `backtest.py` file. Try running the function to plot the largest returns in the dataset. Study the output of the function (i.e. the plot) for `max_rank` in {0, 1, 2, 3} to see the different assets with the largest returns. (Note that we won't study or attempt to change anything past max_rank=3 for the purposes of this class–feel free to explore, though)

[ ]:
```
for i in range(4): plot_asset_with_max_return(returns, prices, max_rank=i)
```


Rank: 0
Ticker: 2269.T
Date: 2009-04-01
Price t: 674.42
Price t-1: 305.32
Return: 120.9%
4000
3000
2000
1000
0
2008
2010
2012
2014
2016
2018
Date


Rank: 1
Ticker: 9201.T
Date: 2010-02-01
Price t: 1.56
Price t-1: 0.78
Return: 100.0%
4000
3000
2000
1000
0
2008
2010
2012
2014
2016
2018
Date





There may be some issues. Let's look into them.

1.1 Part 1

Let's investigate 2269.T, which has the top rank (0). What's going on here? Use the `metadata` and the internet (generative AI ok here–but check the sources!) to find out what's going on. What do you think this price difference might be due to?

[ ]:
```
# Your code here
company = metadata.loc["2269.T"]
print(company)
prices.loc["2009-03-15":"2009-04-05", "2269.T"]
plot_asset_with_max_return(returns, prices, max_rank=0)
```

```
Ticker
Industry                          Foods
Sector                   Consumer Goods
Company       MEIJI.HOLDINGS.CO…LTD.
Name: 2269.T, dtype: object
```

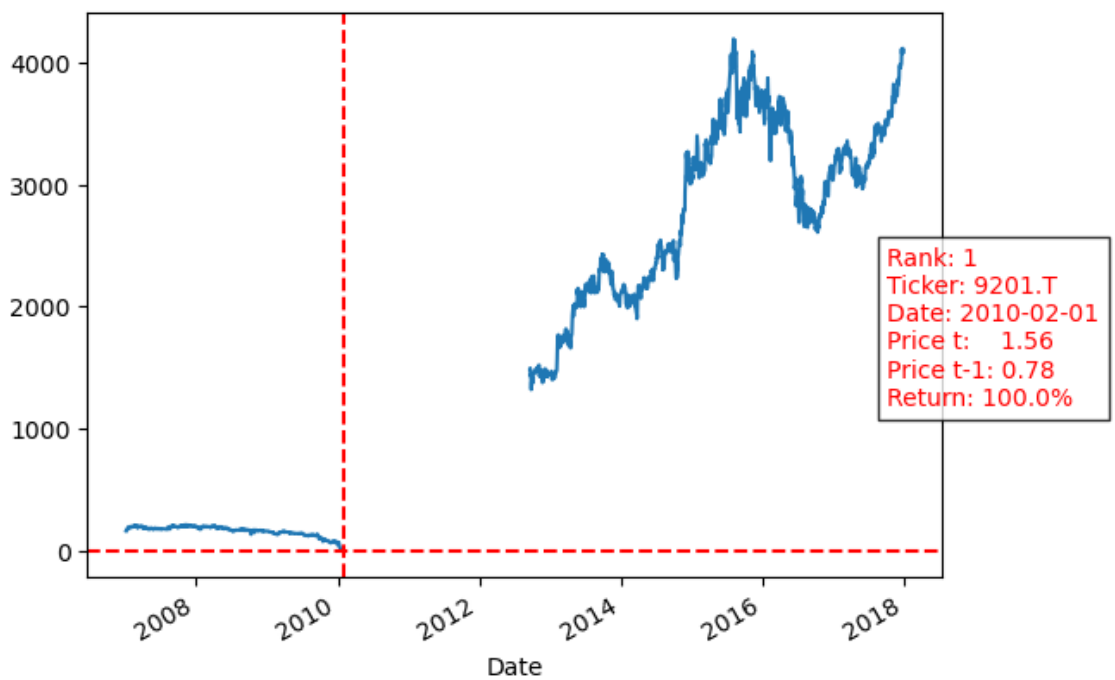
Your answer here: On 2009 April 1st , Meiji Holdings Co., Ltd was established via a finacial reorganization of two companies, Meiji Seika zand Meiji Dairies. This means that when the stock was combined together, the sub-company shares were converted into the shares of the larger company Meiji Holdings. Hence, the price changed/reset as a consequence of this. The stock price on that day reflected this reorganization. So, instead of having an actual stock price increase due to fundemental changes in the company's revenue and behavior stream, it was due to an external reorganization that caused the price to reflect the new status of the combined Meiji Holdings Co.

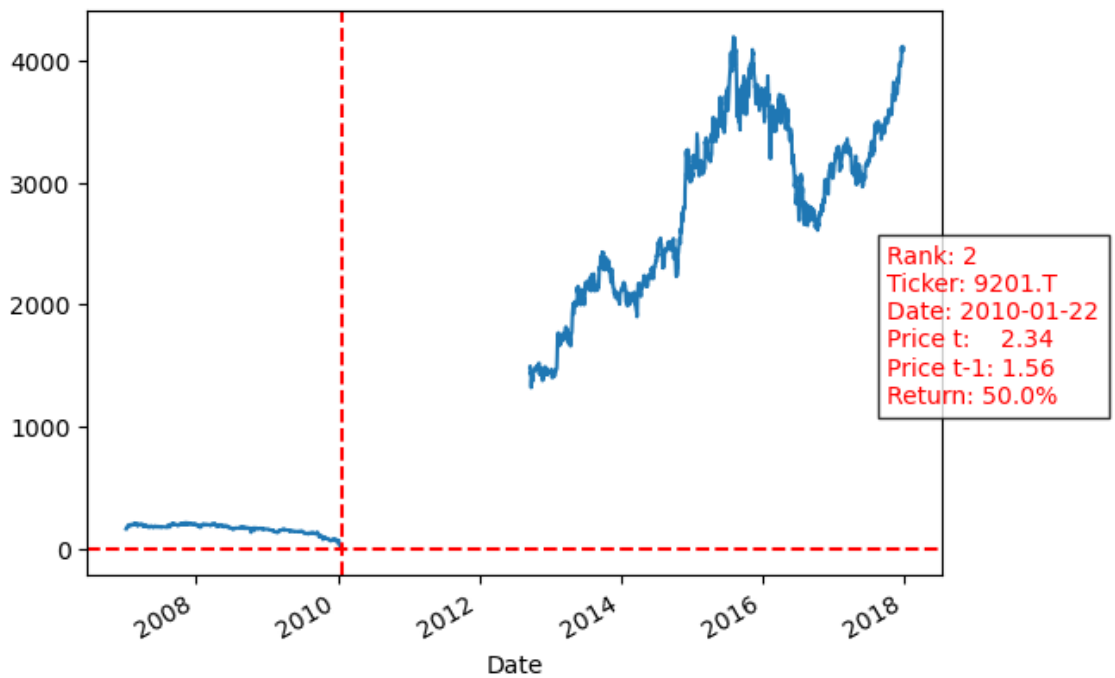
1.2 Part 2

Now let's investigate 9201.T, which has the second and third-highest return. Again, use the metadata and the internet to find out what the price difference (and delisting) might be due to.

[ ]:
```
# Your code here
company_2 = metadata.loc["9201.T"]
print(company_2)
plot_asset_with_max_return(returns, prices, max_rank=1)
plot_asset_with_max_return(returns, prices, max_rank=2)
prices.loc["2010-01-25":"2010-02-05", "9201.T"]
```

```
Ticker
Industry                    Air Transport
Sector      Transportation and Utilities
Company         JAPAN.AIRLINES.CO…LTD.
Name: 9201.T, dtype: object
```





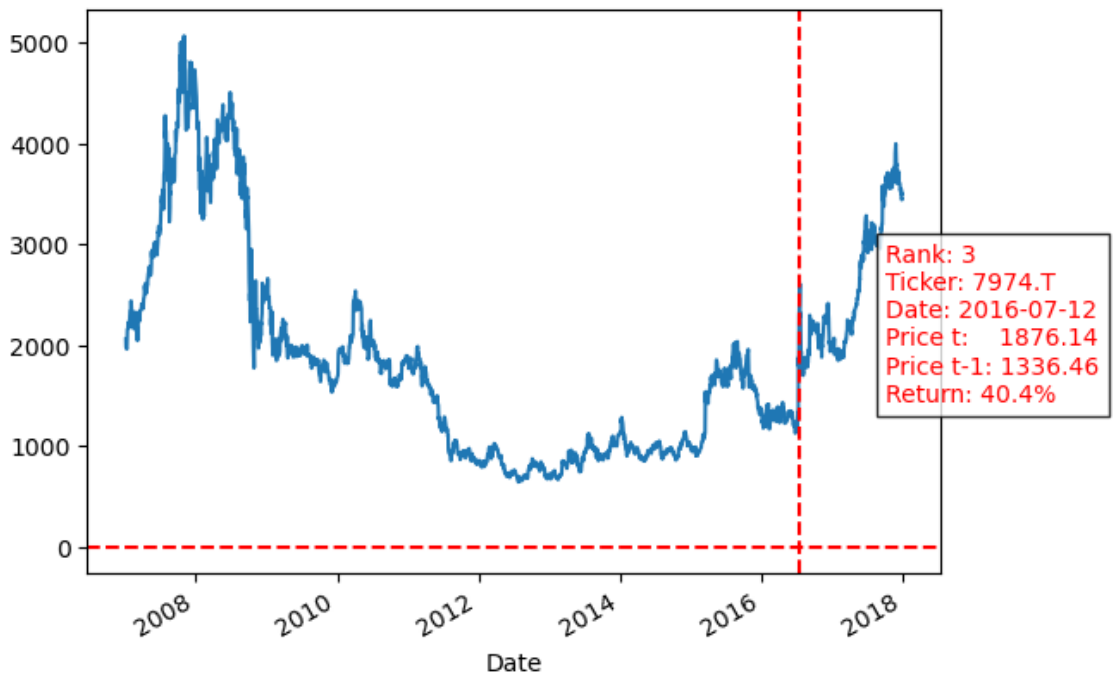
[ ]:
```
Date
2010-01-25    1.5571
2010-01-26    1.5571
2010-01-27    1.5571
2010-01-28    1.5571
2010-01-29    0.7785
2010-02-01    1.5571
2010-02-02    0.7785
2010-02-03    0.7785
2010-02-04       NaN
2010-02-05       NaN
Name: 9201.T, dtype: float64
```

Your answer here: Right around the middle of January, Japanese Airlines filed for bankruptcy. This meant that after January 19th (when they filed title 11), the stock price was practically worthless and near 0. But at the end of January and the start of Feburary, right before the stock was delisted from the stock market, the stock price was alternating widly between 1.5571 and 0.7785, and there were no other values that the stock price was taking on. This means that there was probably some mechanical restructuring of the stock price during the period right before the delisting. The could be credited to, but are not limited to, unit changes, stock consolidations, or stock splits. Hence this period of prices that oscillates between 0.7785 and 1.5571 will consistently show returns that are 100% but are rather uninformative.

1.3 Part 3

Finally, let's investigate 7974.T, which has the third-highest return. Again, find out what might've caused this price difference. Is it different from the others?

[ ]:
```
plot_asset_with_max_return(returns, prices, max_rank=3)
company_3 = metadata.loc["7974.T"]
print(company_3)
prices.loc["2016-07-01":"2016-07-20", "7974.T"]
```




```
Ticker
Industry                Services
Sector            Consumer Goods
Company      NINTENDO.CO…LTD.
Name: 7974.T, dtype: object
```

[ ]:
```
Date
2016-07-01    1204.2137
2016-07-04    1216.5352
2016-07-05    1190.2494
2016-07-06    1181.2137
2016-07-07    1226.8029
2016-07-08    1336.4637
2016-07-11    1336.4637
2016-07-12    1876.1421
2016-07-13    1793.1777
```

```
2016-07-14    2078.2136
2016-07-15    2281.9277
2016-07-19    2609.6772
2016-07-20    2280.6953
Name: 7974.T, dtype: float64
```

Your answer here: On July 6th, 2016, Nintendo relaseed a mobile game Pokemon GO and quickly became popularized across the youth population. The stock price surge on July 12th, captured this sudden surge in popularity of Pokemon GO. Hence on that day, prices rose 40.4 percent from the day before. From a historical point of view, this sudden increase in stock price was not mechanically engineered from stock splits, bankruptcies, or other financial tools. Rather this sudden increase in stock price is a direct consequence of the public's positive sentiment about Nintendo's future earnings.

2 Question 2

(4 points)

Objectively, the best way to deal with erroneous data is to correct it. However, we don't have the ability to do that. The data might have come from an old vendor, a non-responsive government agency, or a human who made errors they weren't aware of (and, they're off on vacation until next month). In these cases, we have a few options: filtering the data (effectively removing the missing data from consideration), imputing the missing values, or using a different data source altogether. Although imputation can be useful for features or signals, it can be a bad idea for asset prices, as prices impact your profit and loss calculations. We'll instead explore the first option: filtering the data.

In this case, we filter the assets. This is known as *asset universe selection*. This is the process of selecting a subset of assets to include for consideration in your analysis, model, or trading. You don't *have* to analyze, trade, etc. these assets, but they'll be in the *universe* of assets that you may want to analyze, model, or trade. Typical asset filters may include minimum number of observations, minimum market cap, minimum liquidity, minimum volume, maximum absolute return, maximum volatility, etc. Here, since we've already restricted ourselves to the large, liquid stocks of the Nikkei index, we don't need those filters. Instead, we'll develop a filter based on data availability and validity.

The file `backtest.py` contains a modified implementation of the Gatev et al. pairs trading strategy. In this HW, we'll finish the implementation and test it. (N.B.: `backtest.py` contains a lot of code. Most of it you don't need to touch, though it's worth reading if you have extra time, as it's a good reference for the code you're writing, and it may be useful for the project.)

In this question, we'll complete one function in it: `select_asset_universe`. In this function, you'll filter the given prices and returns dataframes based on a filter. Your filter should do the following:

1. First, ensure the lookback period is correct. It should be of size `lookback_period` and include dates up to, but not including, the passed `date`.
2. Filter out stocks with any missing prices in the lookback period.
3. Filter out stocks with absolute returns > `max_abs_return` in the lookback period.
4. Return the filtered prices and returns dataframes, along with the tickers of the filtered stocks.

Our implementation is about 12 lines, but it may be done in more or fewer lines.

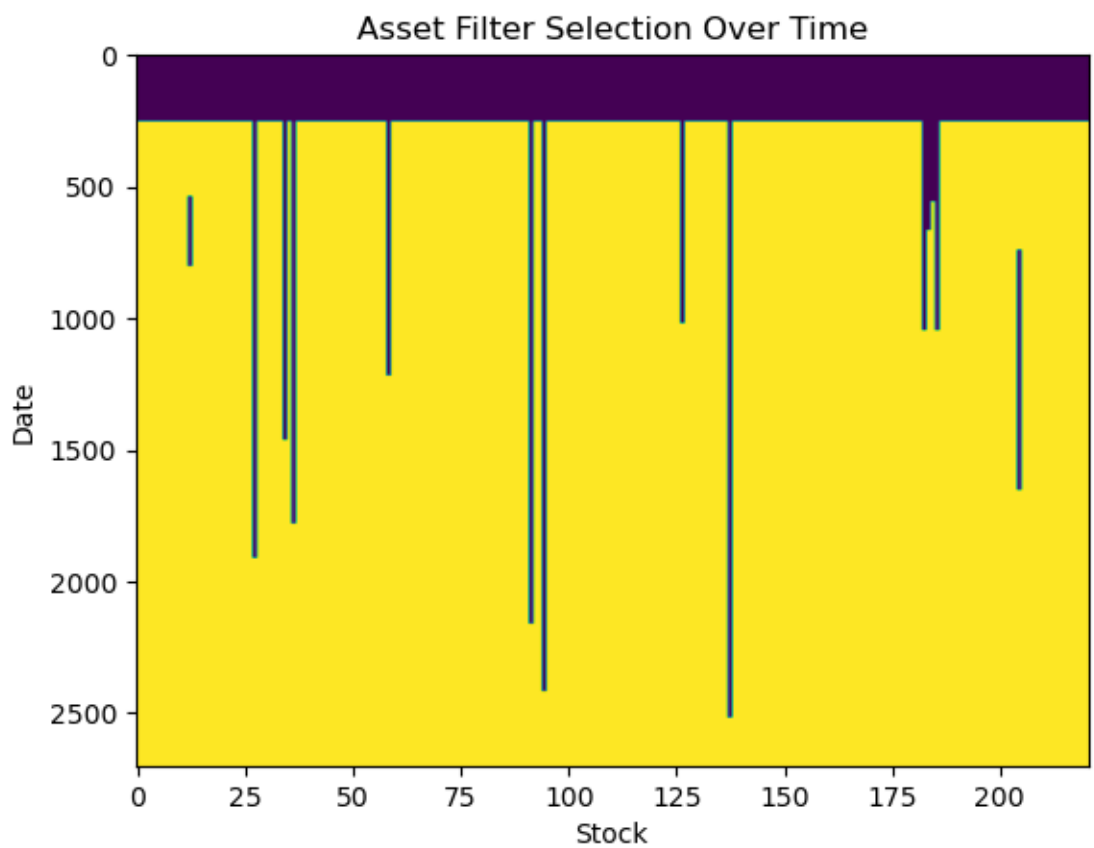
Complete the function in `backtest.py` and paste the completed `select_asset_universe` function below.

[ ]:
```
# Your code here
def select_asset_universe(
    prices: pd.DataFrame,
    returns: pd.DataFrame,
    date: pd.Timestamp,
    config: dict,
) -> Tuple[pd.DataFrame, pd.DataFrame, pd.Index]:
    # Ensure correct lookback period
    lookback_period = config['FORMATION_PERIOD']
    prices, returns = prices.loc[prices.index < date, :], returns.loc[returns.
 ↪index < date, :]
    prices, returns = prices.tail(lookback_period), returns.
 ↪tail(lookback_period)

    # Remove all stocks with missing price data
    filter_tickers = ~prices.isna().any(axis=0)
    prices, returns = prices.loc[:, filter_tickers], returns.loc[:,␣
 ↪filter_tickers]

    # Filter out stocks with too high return
    max_abs_return = config['FILTER_MAX_ABS_RETURN']
    filter_tickers = ~(abs(returns) > max_abs_return).any(axis=0)
    prices, returns = prices.loc[:, filter_tickers], returns.loc[:,␣
 ↪filter_tickers]

    return (prices, returns, prices.columns)
```

Now, let's test it. Execute this cell, which runs your asset filter in a loop and displays your asset filter's selection over time as a boolean matrix. Try different values for `lookback_period` and `max_abs_return` to see how they affect the selection, but finish by running it with `lookback_period=252` and `max_abs_return=0.5`.

[ ]:
```
config = {
    'FILTER_MAX_ABS_RETURN': 0.5,
    'FORMATION_PERIOD': 252,
}

asset_filter = pd.DataFrame(index=prices.index, columns=tickers, data=False)

for date in prices.index[config['FORMATION_PERIOD']:]:
    selected_stocks, selected_rets, selected_uni = select_asset_universe(
        prices,
        returns,
```

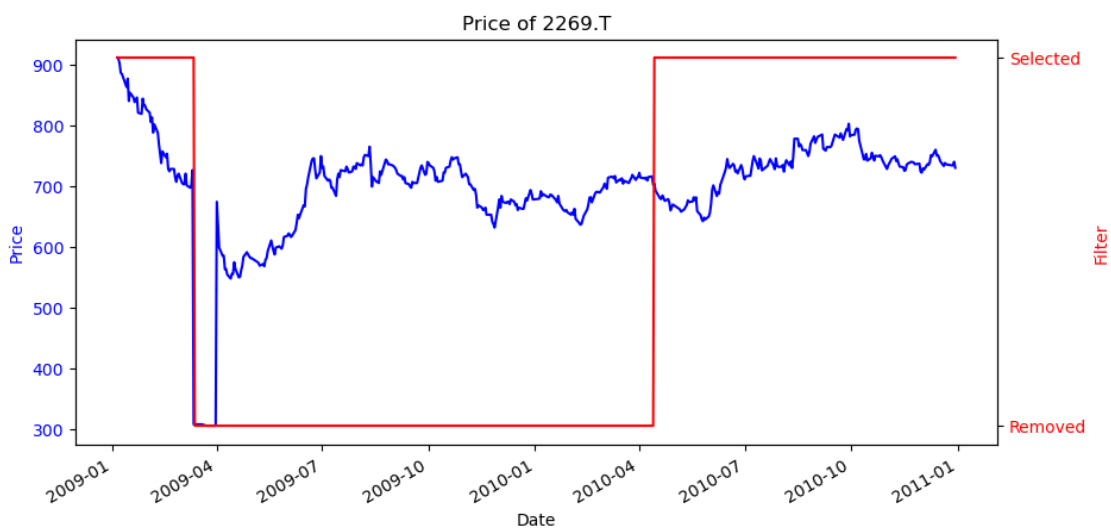
```
        date,
        config=config
    )
    asset_filter.loc[date, selected_stocks.columns] = True

plt.imshow(asset_filter, aspect='auto', cmap='viridis', interpolation=None)
plt.xlabel('Stock')
plt.ylabel('Date')
plt.title('Asset Filter Selection Over Time')
plt.grid(False)
plt.show()
```

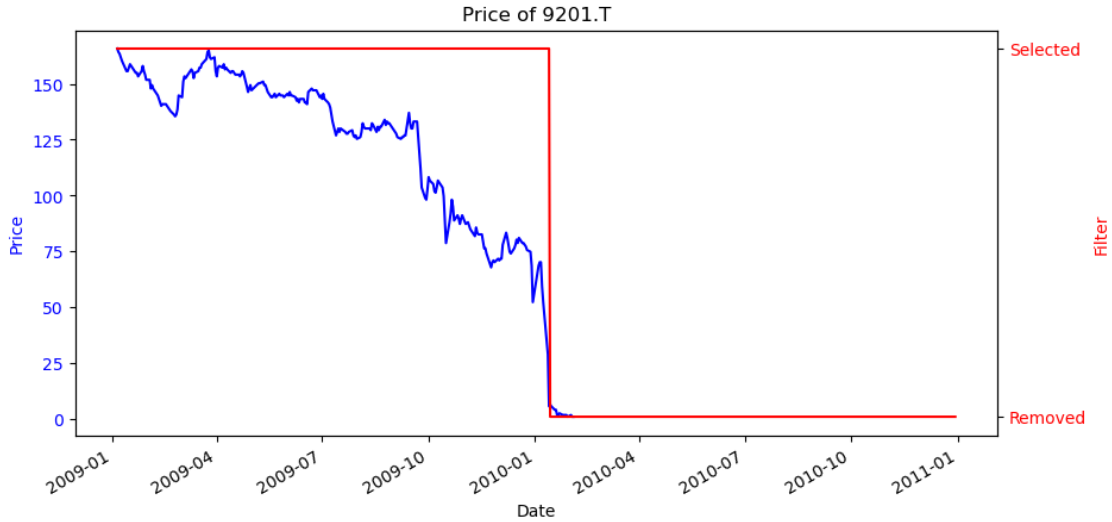


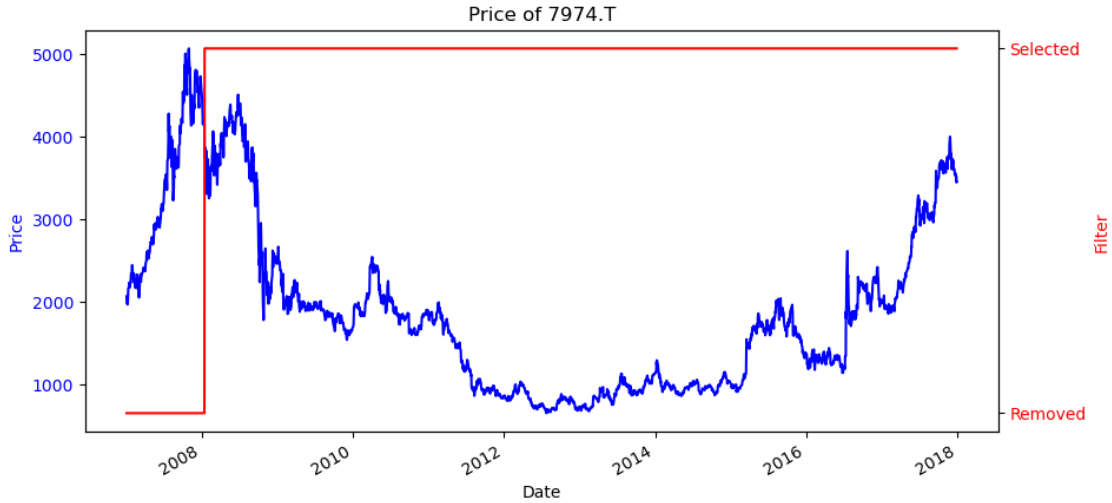
Now let's check to make sure it works properly to avoid selecting stocks with large returns. We'll check the stocks from Question 1 with the largest absolute returns during the lookback period. Run the cell below to see the results.

You should not expect your filter to be perfect, but it should help avoid most of the time associated with the erroneous prices. In a real-world setting, we would want to use more advanced methods to filter out stocks that are not suitable for trading (as well as acquire good data, and avoid any lookahead bias). But for the purposes of the backtests we'll perform in this class, a filter like this suffices.

[ ]:
```
questionable_tickers = [
    ('2269.T', '2009', '2010'),
    ('9201.T', '2009', '2010'),
    ('7974.T', '2007', '2017'),
]

# plot price of each stock and filter status for each stock
for ticker, start, end in questionable_tickers:
    plt.figure(figsize=(10, 5))
    ax1 = plt.gca()
    prices[ticker][start:end].plot(label='Price', color='blue', ax=ax1)
    ax2 = ax1.twinx()
    (1 * asset_filter[ticker][start:end]).plot(label='Filter', color='red',␣
 ↪ax=ax2)
    ax2.set_yticks([0, 1])
    ax2.set_yticklabels(['Removed', 'Selected'])
    plt.title(f'Price of {ticker}')
    ax1.set_ylabel('Price', color='blue')
    ax2.set_ylabel('Filter', color='red')
    ax1.tick_params(axis='y', labelcolor='blue')
    ax2.tick_params(axis='y', labelcolor='red')
    plt.show()
```







3 Question 3

(5 points)

Now we'll continue on with the implementation of the GGR pairs trading strategy. We need to write a function to compute the distances between the stocks from the GGR paper. Your function should do the following:

1. Take in a dictionary with the configuration parameter `NUM_PAIRS`, which is set to an integer such as 20.
2. Normalize the price series of each stock to start at 1.
3. If the `config['DISTANCE_METRIC']` configuration parameter is set to `"ssd"`, compute the

sum of squared differences (SSD) between the normalized price series of each pair of stocks. Otherwise throw an error.

4. Return a dataframe containing the top stock pairs and their corresponding distances.

Feel free to locate the relevant part of the paper if you need more details.

For reference, the return of the `form_pairs` function should look something like this:

stock1	stock2	distance
A	B	0.12
C	D	0.23
E	F	0.40

For extra credit (0.5 points): in addition to the SSD metric, implement your own distance metric which is different from the SSD metric and produces a different ranking. Choose which metric to use based on the `DISTANCE_METRIC` configuration parameter of the `config` argument, which you can set to a string such as `"ssd"` or `"my_metric"`.

Our implementation is fairly naive and constitutes about 16 lines. You can do it in fewer or more lines.

Complete the function in `backtest.py` and paste the complete `form_pairs` function below:

[ ]:
```
# Your code here
def form_pairs(prices: pd.DataFrame, config: dict) -> pd.DataFrame:
    allowed_distance_metrics = ['ssd', 'wsd']
    distance_metric = config.get('DISTANCE_METRIC')
    if distance_metric not in allowed_distance_metrics:
        raise ValueError('Distance metric not allowed')

    prices = prices / prices.iloc[0, :]
    returns = prices.pct_change().dropna()

    data = []
    already_present = []
    for company1 in prices.columns:
        for company2 in prices.columns:
            if company1 == company2 or (company2, company1) in already_present:
                continue
            if distance_metric == 'ssd':
                distance = ((prices.loc[:, company1] - prices.loc[:,␣
↪company2])**2).sum()
            elif distance_metric == 'wsd':
                distance = stats.wasserstein_distance(returns.loc[:, company1],␣
↪returns.loc[:, company2])
            data.append([company1, company2, distance])
            already_present.append((company1, company2))

    pairs = pd.DataFrame(data, columns=['stock1', 'stock2', 'distance'])
```

```
    pairs = pairs.sort_values(by='distance').head(config['NUM_PAIRS']).
↪reset_index(drop=True)
    return pairs
```

3.0.1 Extra credit

For the second distance metric we use the empirical Wasserstein distance (wsd). The wsd often also called the earth mover distance is a metric of distance between two probability distributions. We apply this metric on the return space instead of price space. The wsd is commonly used in machine learning, especially in autoencoders and GANs, and have also found many applications in finance. For instance, Backhoff-Veraguas et al. uses a type of wsd that takes into account the temporal structure of financial time series for hedging purposes. We use a slightly more naive and straightforward wsd that does not account for temporal structure.

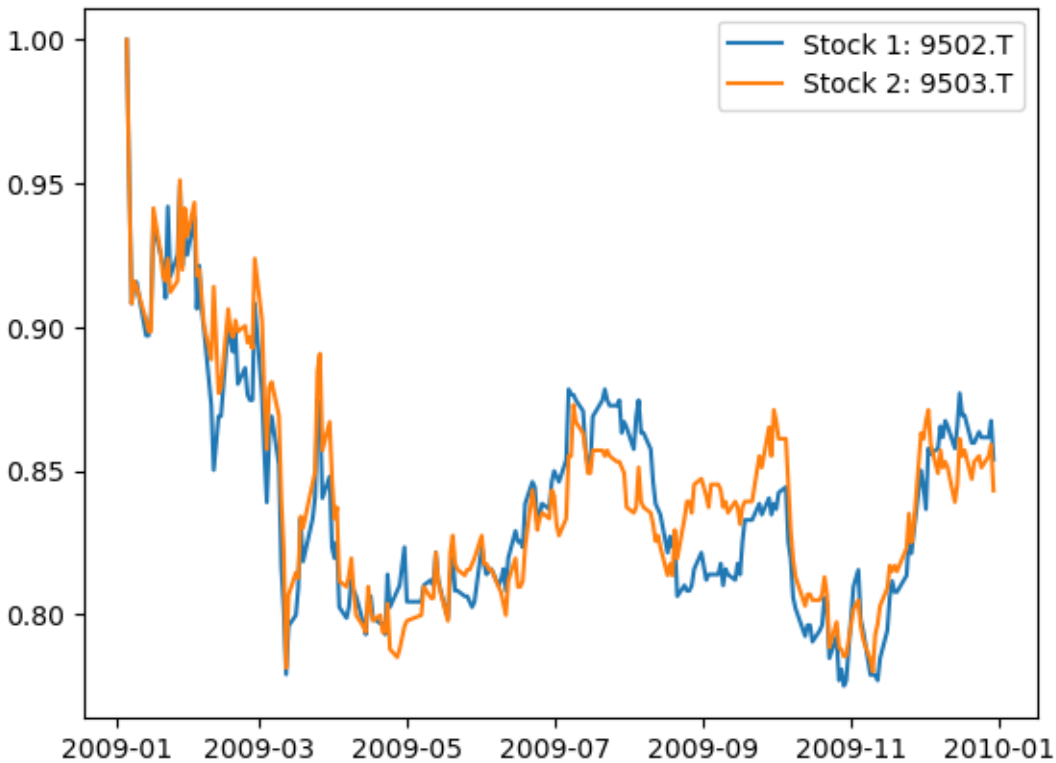
Link to paper: https://link.springer.com/article/10.1007/s00780-020-00426-3

Now, let's test it. Make sure your `form_pairs` function is working correctly by running the code below. We'll compute 5 pairs as a demo. This cell plots the top pair's prices over the lookback window. The prices should look pretty close and cross fairly frequently.

[ ]:
```
# Set a date for the asset filter
date = pd.Timestamp('2010-01-04')
config = {
    'FILTER_MAX_ABS_RETURN': 0.5,
    'FORMATION_PERIOD': 252,
    'NUM_PAIRS': 5,
    'DISTANCE_METRIC': 'ssd',
}

# Filter assets
filtered_prices, filtered_returns, valid_stocks = select_asset_universe(
    prices, returns, date, config=config
)

# Compute distances
pairs = form_pairs(filtered_prices, config)
display(pairs)

# Plot the top pair's normalized prices over the (approximate) lookback window
ticker1 = pairs.loc[0, 'stock1']
ticker2 = pairs.loc[0, 'stock2']

s1 = filtered_prices[date - pd.Timedelta(days=365):date][ticker1]
s2 = filtered_prices[date - pd.Timedelta(days=365):date][ticker2]
s1 = s1 / s1.iloc[0]
s2 = s2 / s2.iloc[0]
plt.plot(s1, label=f'Stock 1: {ticker1}')
```

```
plt.plot(s2, label=f'Stock 2: {ticker2}')
plt.legend()
plt.show()
```

```
   stock1  stock2  distance
0  9502.T  9503.T    0.0516
1  9531.T  9532.T    0.1141
2  9503.T  9532.T    0.1636
3  9021.T  9531.T    0.1701
4  9502.T  9532.T    0.1726
```



4 Question 4

(6 points)

Now we have the ability to create pairs. However, we'll want to do some calculation of hedge ratios and cointegration tests to make sure the pairs are actually cointegrated. Cointegration is not covered in Gatev et al., but may (or may not!) improve the strategy. In any case, we'll apply it here as a filter.

We've written a function called `estimate_hedge_ratio` in `backtest.py`. We'll use that function to estimate the hedge ratios, or *betas*, of the pairs. We'd like you to complete it. Do the following:

1. Go through the `pairs` dataframe and for each pair, extract the price series for the two stocks.

2. Use a method to compute the hedge ratio between the two stocks. If `config['HEDGE_RATIO_METHOD'] == 'ols'`, use OLS regression with no intercept to compute the hedge ratio by regressing stock 2 on stock 1. If `config['HEDGE_RATIO_METHOD'] == 'unit'`, set the hedge ratio to 1. Otherwise, throw an error.
3. Use the hedge ratio to compute the spread $s_t = p_2(t) - \beta p_1(t)$. Store the β's.
4. Perform an ADF test on the spread (the `arch.unitroot` package is useful).
5. Store stock1, stock2, the hedge ratio, the distance, the ADF statistic, the ADF p-value, whether the pair is cointegrated at the `config['COINT_THRESHOLD']` significance level in the results dataframe.
6. Filter the results dataframe to only include pairs with an ADF p-value less than or equal to the threshold.
7. Return the results dataframe.

For reference, the return of the `estimate_hedge_ratio` function may look something like this:

stock1	stock2	hedge_ratio	distance	adf_stat	adf_pvalue	is_cointegrated
A	B	1.00	0.12	-3.56	0.01	$True$
C	D	1.00	0.23	-3.14	0.02	$True$
E	F	1.00	0.40	-2.26	0.04	$True$
E	G	1.00	0.56	-2.75	0.03	$True$

For extra credit (0.5 points): in addition to the OLS and unit hedge ratio methods, implement your method for computing a hedge ratio, which is different from OLS and produces different values. Don't just choose a single integer, such as 2. Then, in your code, choose which regression method to use based on the `HEDGE_RATIO_METHOD` configuration parameter of the `config` argument, which you can set to a string such as `"ols"`, `"unit"`, or `"my_method"` (please choose a more descriptive name than "my_method").

Our implementation is fairly naive and constitutes about 20 lines. You can do it in fewer or more lines.

Complete the function in `backtest.py` and paste your complete `estimate_hedge_ratio` function below:

[ ]:
```
# Your code here
def estimate_hedge_ratio(prices: pd.DataFrame, pairs: pd.DataFrame, config:␣
↪dict) -> pd.DataFrame:
    idxs, hedge_ratios, adf_stats, adf_pvalues, is_cointegrated = [], [], [],␣
↪[], []
    returns = prices.pct_change().dropna()
    for idx, (s1, s2, _) in pairs.iterrows():
        s1_price, s2_price = prices.loc[:, s1], prices.loc[:, s2]
        if config['HEDGE_RATIO_METHOD'] == 'unit':
            hedge_ratio = 1.0
        elif config['HEDGE_RATIO_METHOD'] == 'ols':
            hedge_ratio = sm.OLS(s2_price, s1_price).fit().params.iloc[0]
        elif config['HEDGE_RATIO_METHOD'] == 'tls':
            r1, r2 = returns.loc[:, s1], returns.loc[:, s2]
```

```
            r1_var, r2_var, r1r2_cov = r1.var(), r2.var(), np.cov(r1, r2)[0,1]
            hedge_ratio = (r2_var - r1_var + np.sqrt((r2_var - r1_var)**2 +␣
↪4*r1r2_cov**2)) / (2*r1r2_cov)
        hedge_ratios.append(hedge_ratio)
        spread = s2_price - hedge_ratio*s1_price
        adf_test = ADF(spread)
        adf_stats.append(adf_test.stat)
        adf_pvalues.append(adf_test.pvalue)
        is_cointegrated.append(adf_test.pvalue < config['COINT_THRESHOLD'])
        idxs.append(idx) # not strictly necessary, but more robust to have the␣
↪same idx since we merge on idx
    concat_df = pd.DataFrame(
        data=np.array([hedge_ratios, adf_stats, adf_pvalues, is_cointegrated]).
↪T,
        columns=['hedge_ratio', 'adf_stat', 'adf_pvalue', 'is_cointegrated'],
        index = idxs
    )
    return_df = pd.merge(pairs, concat_df, left_index=True, right_index=True)
    return_df = return_df.loc[return_df['is_cointegrated'] == 1, :].
↪reset_index(drop=True)
    return return_df
```

4.0.1 Extra credit

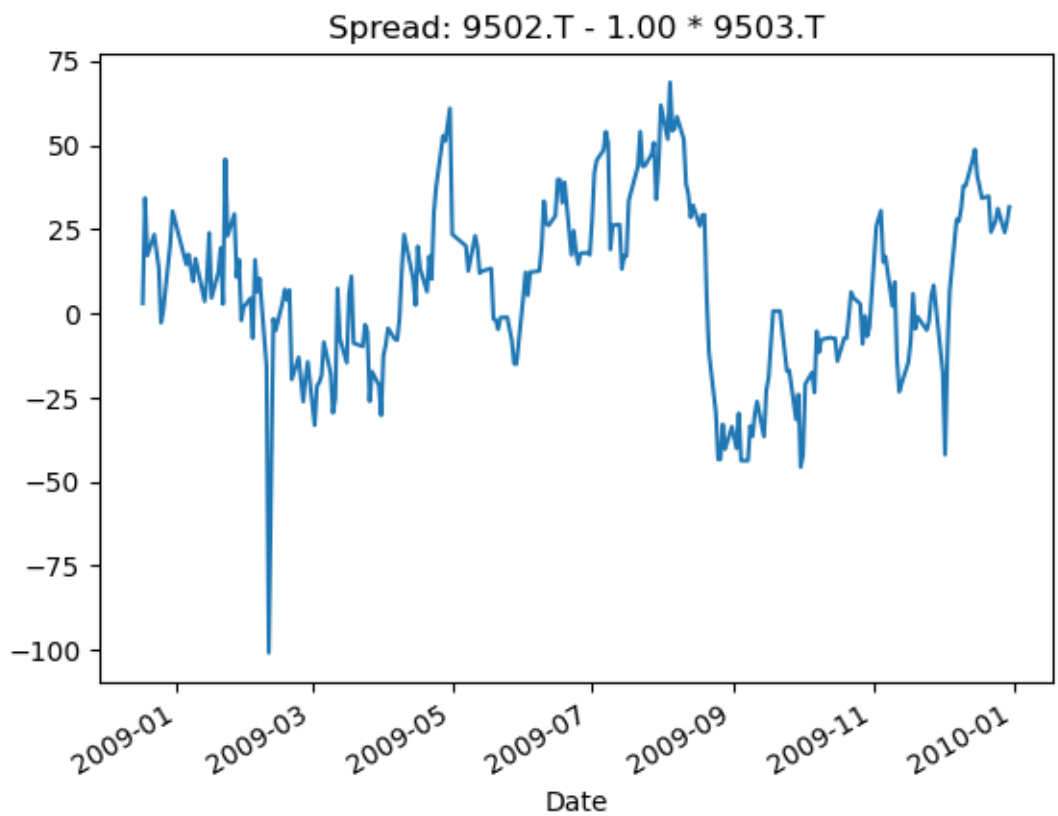
We implement the total least squares (tls) or orthogonal regression as our alternative method. This method has some advantages over ols. For instance it treats the two assets eqaully, one is not modelled as a function of the other in the same way. Another and maybe more important advantage is is that it minimizes spread variance. This method is equivalent to PCA on the pairs. We do this in return space.

Now we'll test your implementation. Run the cell below. The spread you observe should appear to be mostly oscillating around its mean value.

[ ]:
```
# Take the pairs you created above and prices and returns and
# pass them to the `estimate_hedge_ratio` function.

config = {
    'DISTANCE_METRIC': 'ssd',
    'HEDGE_RATIO_METHOD': 'unit',
    'COINT_THRESHOLD': 0.05,
}

# Estimate hedge ratios
hedge_ratios = estimate_hedge_ratio(filtered_prices, pairs, config)
display(hedge_ratios)

# Plot the first hedge ratio as a spread
pair_idx = 0
```

```
ticker1 = hedge_ratios['stock1'].iloc[pair_idx]
ticker2 = hedge_ratios['stock2'].iloc[pair_idx]
hr = hedge_ratios['hedge_ratio'].iloc[pair_idx]
s1 = filtered_prices[ticker1]
s2 = filtered_prices[ticker2]
spread = s1 - hr * s2
spread.plot()
plt.title(f'Spread: {ticker1} - {hr:.2f} * {ticker2}')
plt.show()
```

```
   stock1  stock2  distance  hedge_ratio  adf_stat  adf_pvalue  \
0  9502.T  9503.T    0.0516          1.0   -3.5541      0.0067
1  9503.T  9532.T    0.1636          1.0   -4.0107      0.0014
2  9021.T  9531.T    0.1701          1.0   -3.1722      0.0217
3  9502.T  9532.T    0.1726          1.0   -2.9486      0.0400

   is_cointegrated
0              1.0
1              1.0
2              1.0
3              1.0
```



5 Question 5

(8 points)

Now that we have identified cointegrated pairs and estimated their hedge ratios, we need to generate trading signals based on the divergence and convergence of these pairs. This is a critical component of the Gatev et al. pairs trading strategy.

We've created a function called `compute_signal` in `backtest.py`. Your task is to implement this function according to the Gatev et al. methodology. Complete the following steps:

1. Loop through each pair in the `hedge_ratios` dataframe.
2. Extract the price series for both stocks in each pair from the `prices` dataframe.
3. Calculate the spread between the two stocks using the hedge ratio. The spread should be calculated as: `spread = price2 - hedge_ratio * price1`
4. Implement a rolling window calculation to standardize the spread:
   - Calculate the rolling mean of the spread using a window size of `config['ESTIMATION_PERIOD']`
   - Calculate the rolling standard deviation using the same window size
   - Compute the z-score: `z_score = (spread - rolling_mean) / rolling_std`
5. Initialize an empty signal dataframe for the pair.
6. Loop through time for the pair, generating trading signals based on the z-score and price convergence:
   - **Entry Conditions**:
     - If z-score < `-config['Z_THRESHOLD']`, enter a long spread position (signal = 1)
     - If z-score > `config['Z_THRESHOLD']`, enter a short spread position (signal = -1)
   - **Exit Conditions**:
     - According to Gatev et al., positions should be closed when the normalized prices cross (converge)
     - You need to track which normalized price was higher/lower at trade entry
     - Close the position (signal = 0) when the normalized prices cross back over each other
7. For each pair, store the following in the signals dictionary:
   - stock1 and stock2 identifiers
   - hedge_ratio
   - signal series
   - z_score series
   - any additional metadata you find useful e.g. for debugging, such as series for the spread, crosses, mean, volatility, etc.
8. Return the completed signals dictionary.

5.1 Important Implementation Details

- You must properly track the state of each trade (whether you're in a trade or not)
- You need to record which normalized price was higher/lower at trade entry to determine when price convergence occurs
- The signals should be a time series with the same index as the price data
- For each date, the signal should be:
  - 1: Long spread (short stock1, long stock2)
  - -1: Short spread (long stock1, short stock2)
  - 0: No position

5.2 Example Output

The returned dictionary should have keys for each pair and values containing the signal information:

```
{
    'STOCK1_STOCK2': {
        'stock1': 'STOCK1',
        'stock2': 'STOCK2',
        'hedge_ratio': 1.25,
        'signal': pd.Series(
            data=[0, 0, 1, 1, 0, -1, -1, 0, ...],
            index=pd.DatetimeIndex([
                '2009-01-01', '2009-01-02', '2009-01-03',
                '2009-01-04', '2009-01-05', '2009-01-08',
                '2009-01-09', '2009-01-10', ...
            ])
        ),

        'z_score': pd.Series(
            data=[0.5, 1.8, 2.2, 1.9, 0.8, -1.2, -2.1, -1.8, ...],
            index=pd.DatetimeIndex([
                '2009-01-01', '2009-01-02', '2009-01-03',
                '2009-01-04', '2009-01-05', '2009-01-08',
                '2009-01-09', '2009-01-10', ...
            ])
        ),
        # Add any additional metadata you find useful
    },
    # Additional pairs...
}
```

Our implementation is about 55 lines, but this can be done in more or fewer lines.

Complete the function in `backtest.py` and paste the complete `compute_signal` function below:

[ ]:
```python
# # Your code here

def compute_signal(prices, hedge_ratios, config):
    signals = {}
```

```python
for _, row in hedge_ratios.iterrows():
    s1 = row["stock1"]
    s2 = row["stock2"]
    hr = row["hedge_ratio"]

    price1 = prices.loc[:, s1]
    price2 = prices.loc[:, s2]

    spread = price2 - hr * price1

    window_size = config["ESTIMATION_PERIOD"]
    rolling_mean = spread.rolling(window = window_size).mean()
    rolling_stdev = spread.rolling(window = window_size).std()

    rolling_z = (spread - rolling_mean) / rolling_stdev

    norm_p1 = price1 / price1.iloc[0]
    norm_p2 = price2 / price2.iloc[0]

    signal = pd.Series(0, index = prices.index, dtype = int)
    trade = False
    entry_signal = 0
    p2_higher = None

    for t in range(len(prices)):
        z = rolling_z.iloc[t]

        if not trade:
            if pd.isna(z):
                signal.iloc[t] = 0
            elif z < -config["Z_THRESHOLD"]:
                trade = True
                entry_signal = 1
                if norm_p2.iloc[t] > norm_p1.iloc[t]:
                    p2_higher = True
                else:
                    p2_higher = False
                signal.iloc[t] = 1
            elif z > config["Z_THRESHOLD"]:
                trade = True
                entry_signal = -1
                if norm_p2.iloc[t] > norm_p1.iloc[t]:
                    p2_higher = True
                else:
                    p2_higher = False
                signal.iloc[t] = -1
```

```
                else:
                    signal.iloc[t] = 0

            else:
                signal.iloc[t] = entry_signal

                if p2_higher and norm_p2.iloc[t] <= norm_p1.iloc[t]:
                    trade = False
                    entry_signal = 0
                    p2_higher = None
                    signal.iloc[t] = 0
                elif not p2_higher and norm_p2.iloc[t] >= norm_p1.iloc[t]:
                    trade = False
                    entry_signal = 0
                    p2_higher = None
                    signal.iloc[t] = 0

        signals[f"{s1}_{s2}"] = {
            "stock1": s1,
            "stock2": s2,
            "hedge_ratio": hr,
            "signal": signal,
            "z_score": rolling_z
        }

    return signals
```

Now we'll test your implementation. Run the cell below.

[ ]:
```
# Take the pairs you created above and prices and returns and
# pass them to the `estimate_hedge_ratio` function.

config = {
    'Z_THRESHOLD': 2,
    'ESTIMATION_PERIOD': 20,
    'EXIT_STRATEGY': 'convergence',
}

# Compute signals
signals = compute_signal(filtered_prices, hedge_ratios, config)
pair_idx = 0
```

```python
# Get ticker info for the selected pair
ticker1 = hedge_ratios['stock1'].iloc[pair_idx]
ticker2 = hedge_ratios['stock2'].iloc[pair_idx]
hr = hedge_ratios['hedge_ratio'].iloc[pair_idx]

# Create figure with two subplots sharing x-axis
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex=True)

# Plot normalized prices on first subplot
s1 = filtered_prices[ticker1]
s2 = filtered_prices[ticker2]
s1_norm = s1 / s1.iloc[0]
s2_norm = s2 / s2.iloc[0]
ax1.plot(s1_norm.index, s1_norm, label=ticker1)
ax1.plot(s2_norm.index, s2_norm, label=ticker2)
ax1.set_title('Normalized Price')
ax1.legend()

# Plot spread and signals on second subplot
spread = s1 - hr * s2
ax2.plot(spread.index, spread, label='Spread')

# Add signals to the second subplot
pair_signals = signals[f"{ticker1}_{ticker2}"]
entry_long = np.where(pair_signals['signal'] == -1)[0]
entry_short = np.where(pair_signals['signal'] == 1)[0]
exit_points = np.where(pair_signals['signal'] == 0)[0]

ax2.scatter(spread.index[entry_long], spread.iloc[entry_long],
            color='green', marker='^', s=100, label='Long Position')
ax2.scatter(spread.index[entry_short], spread.iloc[entry_short],
            color='red', marker='v', s=100, label='Short Position')
ax2.scatter(spread.index[exit_points], spread.iloc[exit_points],
            color='black', marker='o', s=50, label='Flat Position')
ax2.set_title(f'Spread and Signals: {ticker1} - {hr:.2f} * {ticker2}')

ax2.legend()

plt.tight_layout()
plt.show()
```

6 Question 6

(8 points)

Last step of the implementation! Now we're going to convert our set of signals for each pair into positions using the `allocate_positions` function. This function will take a set of signals for each pair and allocate the available capital to each pair based on the signals for the given date. The function will then return a set of positions for each stock for the given date. We definitely don't claim this to be the best way to allocate among the pairs, but it's a simple baseline.

Let's get into it:

1. Initialize an empty dictionary to store positions for each stock.
2. Add a few checks to handle edge cases:
   - If the current date is not in the price data, return empty positions
   - If portfolio cash is None or zero/negative, return empty positions
3. Collect active pairs for the current date:

- Iterate through the signals dictionary
- For each pair, extract stock1, stock2, and the hedge ratio
- Skip pairs where we don't have price data for both stocks
- Get the signal with a one-day lag (yesterday's signal for today's execution)
- Skip signals that aren't for the current date or are zero/NaN
- Store active pairs with their relevant information (stocks, hedge ratio, signal value, current prices)

4. If no active pairs are found, return empty positions.
5. Allocate capital equally across all active pairs:
   - Calculate `capital_per_pair = (portfolio_cash * config['MAX_LEVERAGE']) / number of active pairs`
6. For each active pair:
   - Initialize positions for both stocks if they don't exist in the positions dictionary
   - Calculate balanced position sizes using a notional ratio approach:
     - `notional_ratio = (hedge_ratio * price2) / price1`
     - `shares1 = capital_per_pair / (price1 * (1 + notional_ratio))`
     - `shares2 = shares1 * hedge_ratio`
   - Update positions based on signal:
     - `positions[stock1] += -signal_value * shares1`
     - `positions[stock2] += signal_value * shares2`
7. Return the positions dictionary.

The `allocate_positions` function takes the following parameters:

- `signals`: A dictionary with signal information for each pair. Note that this contains signals for the entire trading period, not just the current date. Make sure to avoid using signals from the future!
- `config`: Configuration parameters, containing MAX_LEVERAGE
- `prices`: DataFrame with stock prices for the entire trading period (dates as index, tickers as columns)
- `date`: Current date for price and signal reference
- `portfolio_cash`: Available cash in the portfolio

It returns a dictionary mapping stock names to position sizes (number of shares).

Our implementation is fairly naive and requires about 40 lines of code. You can implement it in fewer or more lines.

Complete the function in `backtest.py` and paste the complete `allocate_positions` function below:

[ ]:
```
# Your code here
def allocate_positions(
    signals: dict,
    config: dict,
    prices: pd.DataFrame,
    date: pd.Timestamp,
```

```
    portfolio_cash: float,
) -> dict:
    positions = {}
    active_pairs = {}
    if date not in prices.index:
        return positions
    # if date == prices.index[0]:
    #     return positions # cannot make trade the first day as we rely on␣
↪lagged signal
    if portfolio_cash == None or portfolio_cash <= 0:
        return positions

    for _, data in signals.items():
        s1, s2, hedge_ratio, signal_series = data['stock1'], data['stock2'],␣
↪data['hedge_ratio'], data['signal']
        try:
            p1, p2 = prices.loc[date, s1], prices.loc[date, s2]
        except KeyError:
            continue
        lagged_signal = signal_series.shift(1).loc[date]
        if np.isnan(lagged_signal) or lagged_signal == 0: continue
        active_pairs[f'{s1}_{s2}'] = {
            'stock1': s1,
            'stock2': s2,
            'price1': p1,
            'price2': p2,
            'hedge_ratio': hedge_ratio,
            'lagged_signal': lagged_signal
        }

    if len(active_pairs) == 0: return positions

    capital_per_pair = (portfolio_cash*config['MAX_LEVERAGE'])/len(active_pairs)
    for _, pair_data in active_pairs.items():
        s1, s2, p1, p2, hedge_ratio, lagged_signal = pair_data.values()
        if not positions.get(s1): positions[s1] = 0
        if not positions.get(s2): positions[s2] = 0
        notional_ratio = hedge_ratio*p2/p1
        shares1 = capital_per_pair/ (p1*(1 + notional_ratio))
        shares2 = shares1*hedge_ratio
        positions[s1] += -lagged_signal*shares1
        positions[s2] += lagged_signal*shares2

    return positions
```

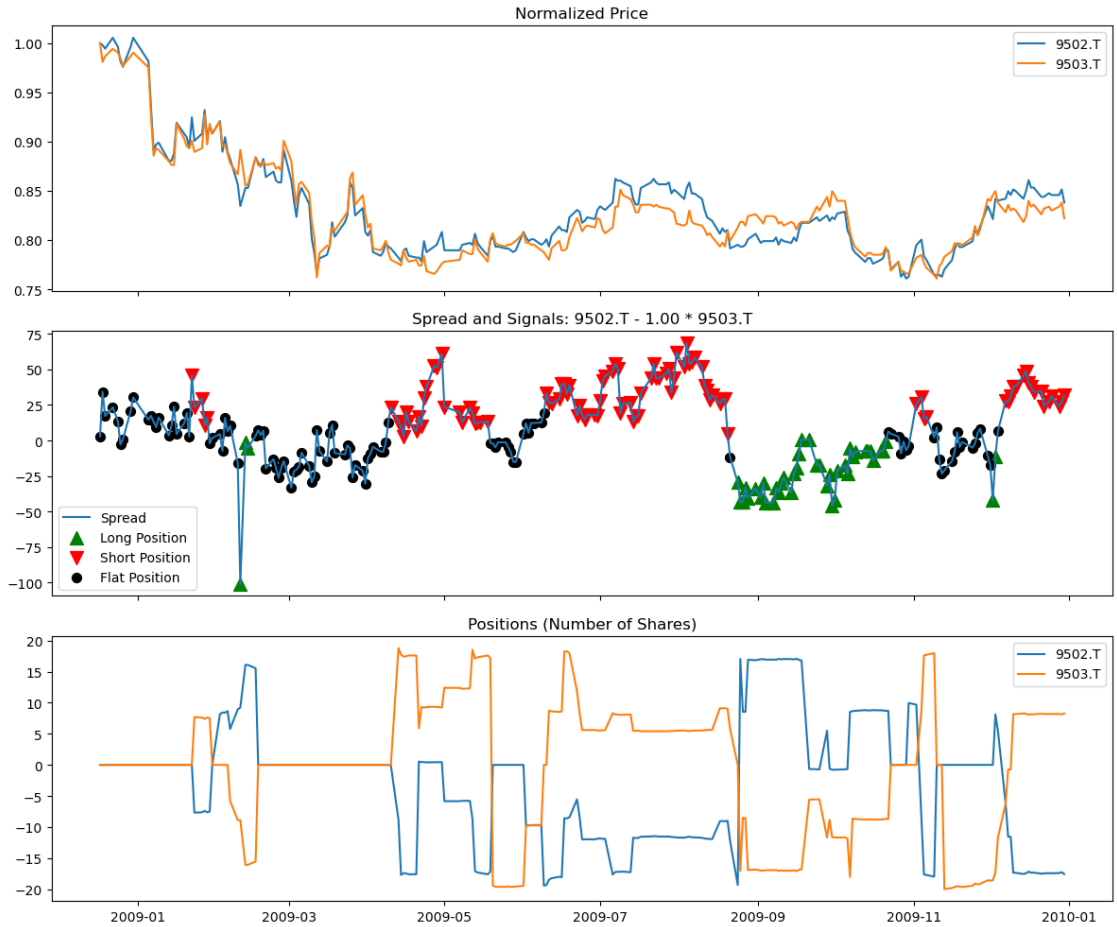
Now, we'll test your implementation. Run the following cell and observe the positions. They should appear to follow most of the signals, more or less. Recall that any single stock may be involved

in multiple pairs, and so the positions you observe here may not be perfectly correlated with the signals, and may not be perfectly offset at all times. This is expected behavior.

[ ]:
```
config = {
    'MAX_LEVERAGE': 0.05,
}

pair_idx = 0

# Get ticker info for the selected pair
ticker1 = hedge_ratios['stock1'].iloc[pair_idx]
ticker2 = hedge_ratios['stock2'].iloc[pair_idx]
hr = hedge_ratios['hedge_ratio'].iloc[pair_idx]

# Compute the positions using allocate_positions
positions_t1 = {}
positions_t2 = {}
for date in filtered_prices.index:
    positions_t1[date] = allocate_positions(
        signals, config, filtered_prices, pd.Timestamp(date), 1_000_000
    ).get(ticker1, 0)
    positions_t2[date] = allocate_positions(
        signals, config, filtered_prices, pd.Timestamp(date), 1_000_000
    ).get(ticker2, 0)

positions = pd.DataFrame(
    {ticker1: positions_t1.values(), ticker2: positions_t2.values()},
    index=filtered_prices.index,
)
display(positions)

# Create figure with three subplots sharing x-axis
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 10), sharex=True)

# Plot normalized prices on first subplot
s1 = filtered_prices[ticker1]
s2 = filtered_prices[ticker2]
s1_norm = s1 / s1.iloc[0]
s2_norm = s2 / s2.iloc[0]
ax1.plot(s1_norm.index, s1_norm, label=ticker1)
ax1.plot(s2_norm.index, s2_norm, label=ticker2)
ax1.set_title('Normalized Price')
ax1.legend()

# Plot spread and signals on second subplot
spread = s1 - hr * s2
ax2.plot(spread.index, spread, label='Spread')
```

```python
# Add signals to the second subplot
pair_signals = signals[f"{ticker1}_{ticker2}"]
entry_long = np.where(pair_signals['signal'] == -1)[0]
entry_short = np.where(pair_signals['signal'] == 1)[0]
exit_points = np.where(pair_signals['signal'] == 0)[0]

ax2.scatter(spread.index[entry_long], spread.iloc[entry_long],
            color='green', marker='^', s=100, label='Long Position')
ax2.scatter(spread.index[entry_short], spread.iloc[entry_short],
            color='red', marker='v', s=100, label='Short Position')
ax2.scatter(spread.index[exit_points], spread.iloc[exit_points],
            color='black', marker='o', s=50, label='Flat Position')
ax2.set_title(f'Spread and Signals: {ticker1} - {hr:.2f} * {ticker2}')
ax2.legend()

# Now plot the positions on the third subplot
ax3.plot(positions.index, positions[ticker1], label=ticker1)
ax3.plot(positions.index, positions[ticker2], label=ticker2)
ax3.set_title('Positions (Number of Shares)')
ax3.legend()

plt.tight_layout()
plt.show()
```

```
             9502.T  9503.T
Date
2008-12-17   0.0000  0.0000
2008-12-18   0.0000  0.0000
2008-12-19   0.0000  0.0000
2008-12-22   0.0000  0.0000
2008-12-24   0.0000  0.0000
…               …       …
2009-12-24 -17.4070  8.1735
2009-12-25 -17.4471  8.2018
2009-12-28 -17.4027  8.1827
2009-12-29 -17.2595  8.1363
2009-12-30 -17.5831  8.2778

[252 rows x 2 columns]
```



7 Question 7

(1 point)

Finally, we'll run a backtest. Run the cell below to see the results of the strategy. It should take about 3 minutes. Note that the results will be written to disk after the backtest is complete, so if you run it again with the same `config` settings, it will load the data and produce a performance analysis in just a few seconds. If you want the backtest to re-run instead of load data, make sure to delete the `results` directory it will create.

Don't be surprised if the results are poor–this basic style of pairs trading does not typically yield significant returns or high Sharpe ratios anymore. However, we've still learned a lot from it, and (significant) variations of pairs trading are still used in industry. The intuition behind the strategy is still sound. Particular implementations of the idea (such as those widely publicized in academic papers), however, will often lose efficacy over time as other market participants "trade away" the particular signals, aka edge, that the strategy exploits. This is known as *alpha decay*.

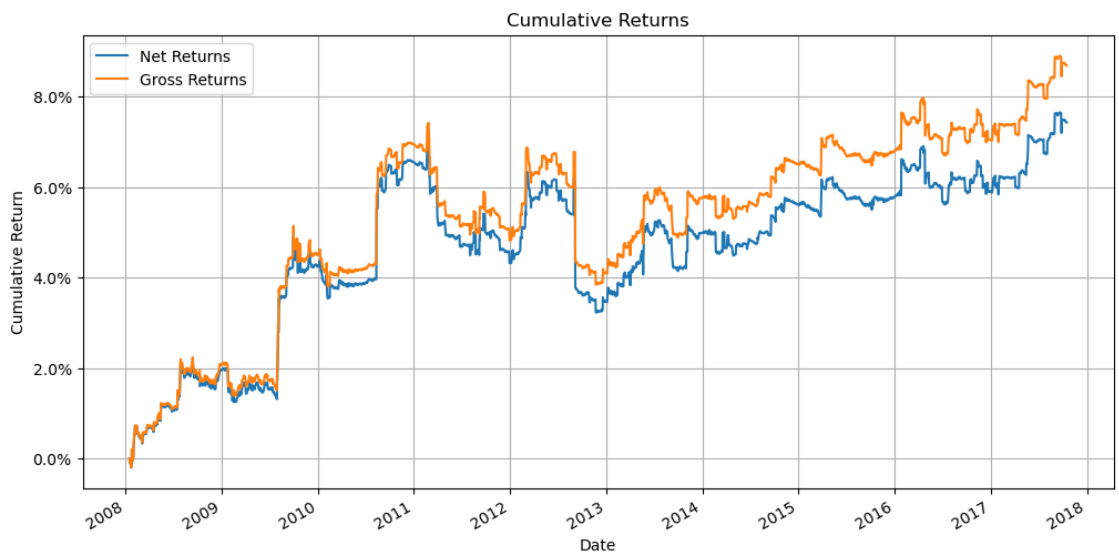
More importantly, don't rush into conclusions about the performance of the strategy if it looks good to you. This is a slightly short backtest over a set of equities which was selected in a way that

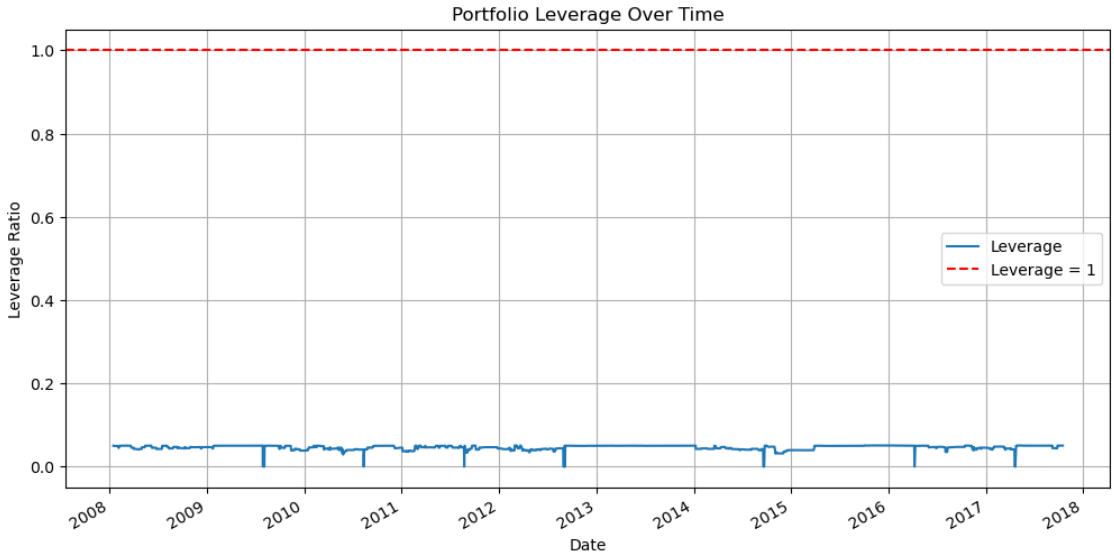
is affected by lookback bias (recall from HW1 that we only selected stocks which were in existence in one particular year of the Nikkei 225). We've also neglected to model a few important things, such as the price impact and bid-ask spread. And most importantly, we've made a dangerously optimistic assumption: that we have the ability to short all of these assets at any time. Japan has several restrictions on shorting, and a broker may either prevent you from borrowing some of these assets, or may *recall* your borrowed asset (perhaps in the middle of a trade!) if the owner wants to sell it. This is a very significant issue, and can turn a profitable-looking strategy into a losing one in reality. Because we don't have historical data on Japanese shorting restrictions (or proxies for it which Gatev et al. 2006 use, such as volume), we've neglected to model this risk in this assignment. For these reasons, it's likely that today, any positive alpha from this strategy is substantially smaller than it appears to be, and probably negligible–or even negative.

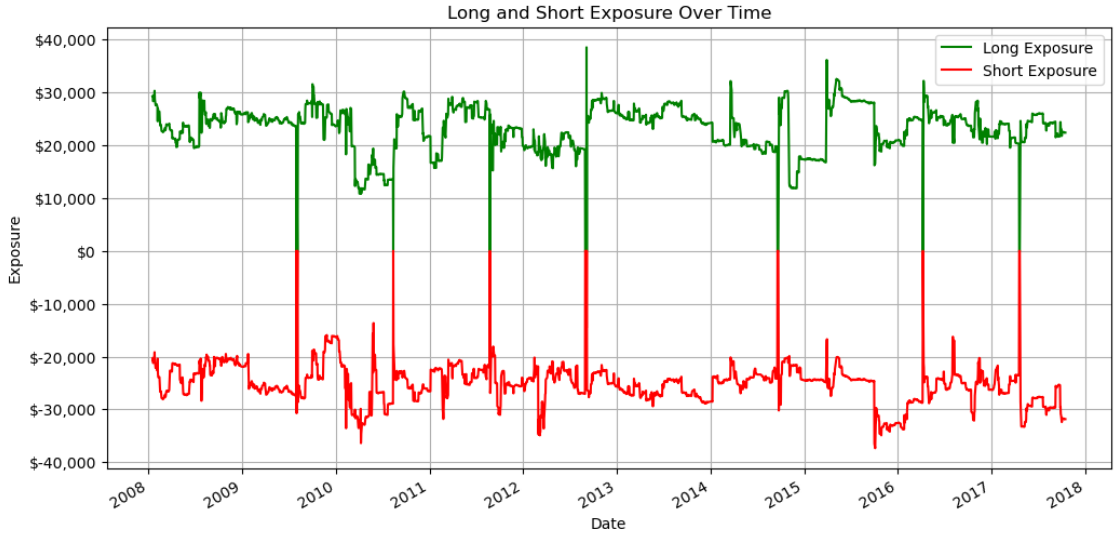
All that aside, take a look at the results and briefly comment on what you observe in the test below. Try to explore or explain why the strategy has performed the way it has. This is an open-ended question, and there is no right answer.

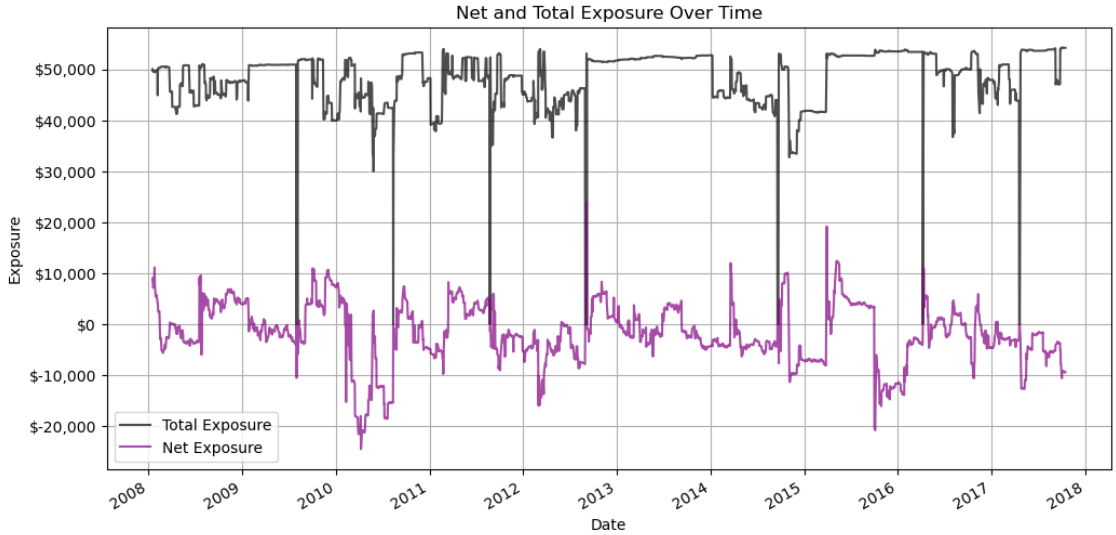
[ ]:
```
(prices, returns, tickers, metadata, portfolio_history, metrics_df, pairs_df) =␣
↪run_backtest()
```

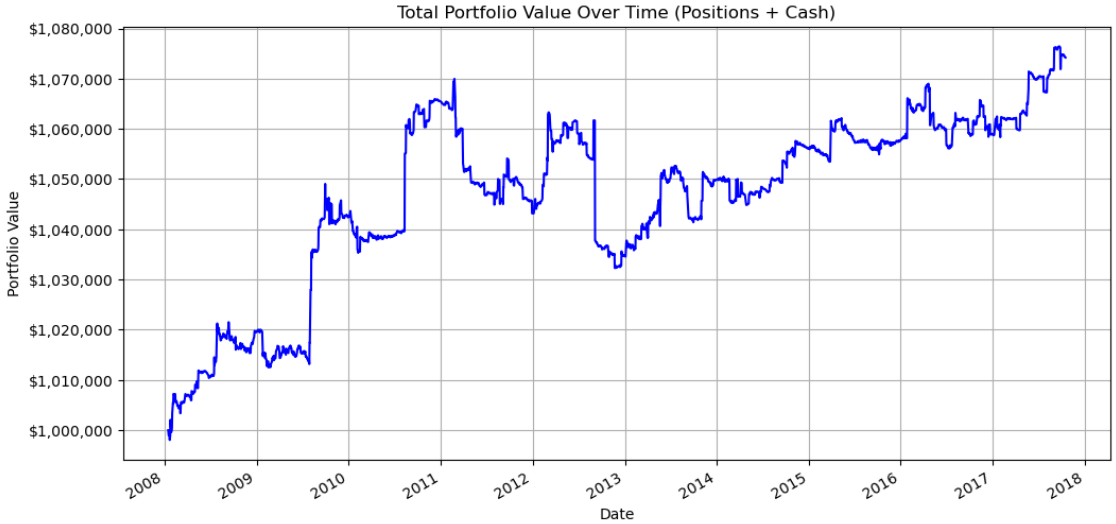
```
Starting from '/Users/kadenwu/Desktop/School/Grad/Year 1/Winter_26/Statistical
Arbitrage/Statistical_Arbitrage_Stanford_MS-E244/Homework 2'
Generated unique ID: closely-hardy-mole
Results already exist for 'closely-hardy-mole'; loading
```

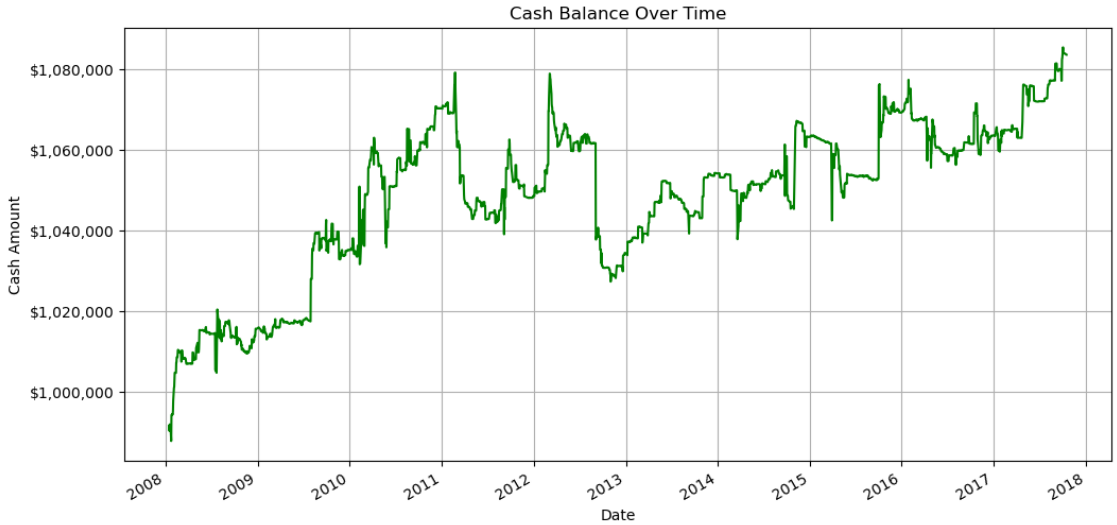


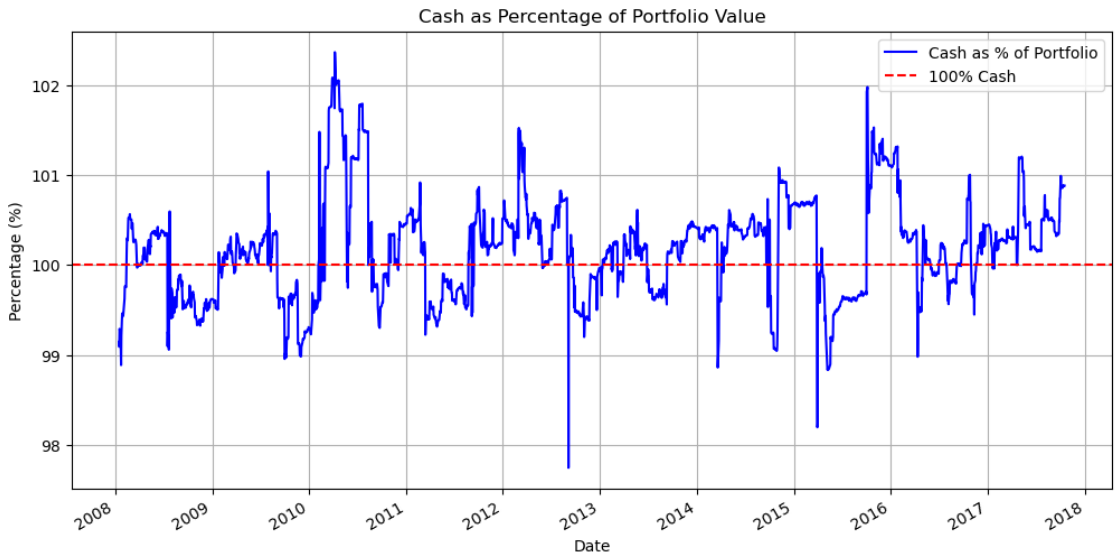

Portfolio Leverage Over Time
Leverage Ratio
1.0
0.8
0.6
0.4
0.2
0.0
Leverage
Leverage = 1
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date

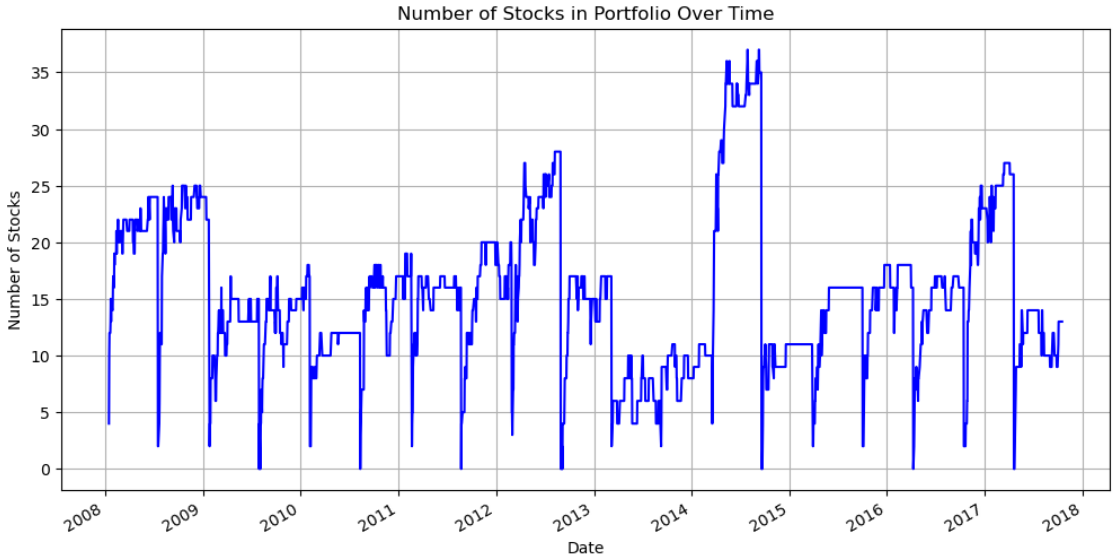

Long and Short Exposure Over Time
Exposure
$40,000
$30,000
$20,000
$10,000
$0
$-10,000
$-20,000
$-30,000
$-40,000
Long Exposure
Short Exposure
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date

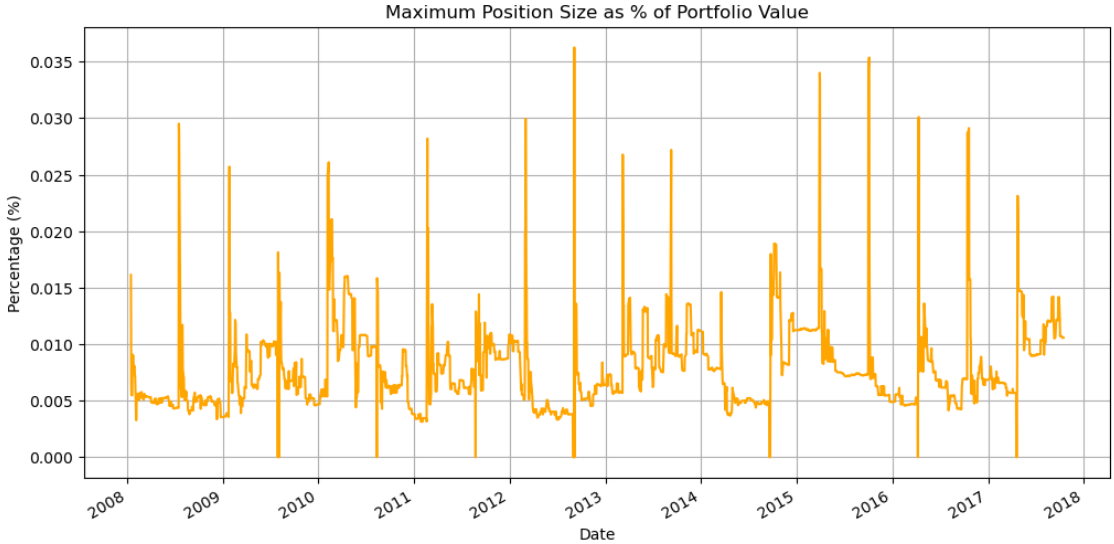

Net and Total Exposure Over Time
$50,000
$40,000
$30,000
$20,000
$10,000
$0
$-10,000
$-20,000
Exposure
Total Exposure
Net Exposure
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date

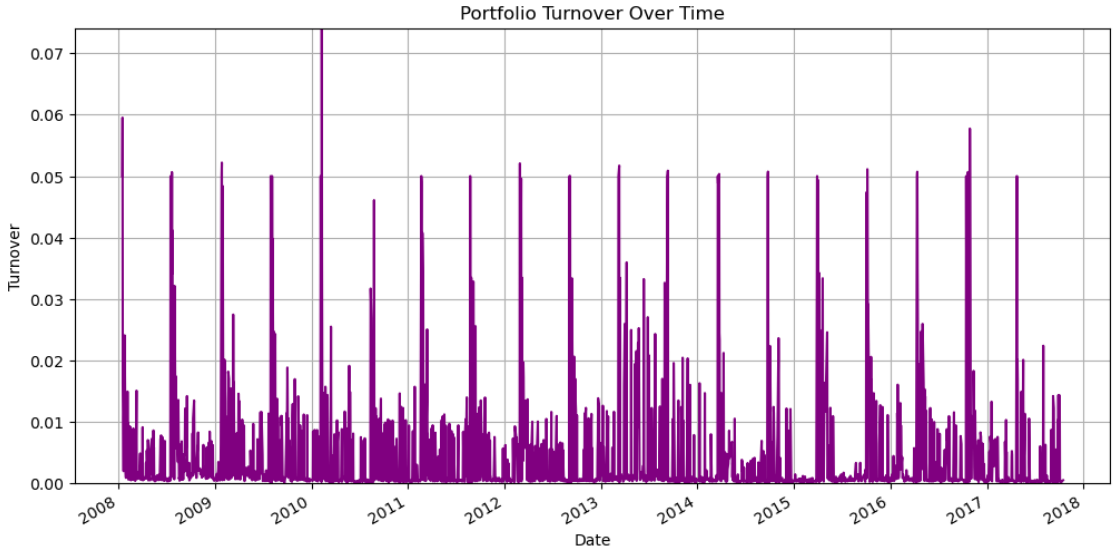

Total Portfolio Value Over Time (Positions + Cash)
$1,080,000
$1,070,000
$1,060,000
$1,050,000
$1,040,000
$1,030,000
$1,020,000
$1,010,000
$1,000,000
Portfolio Value
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date

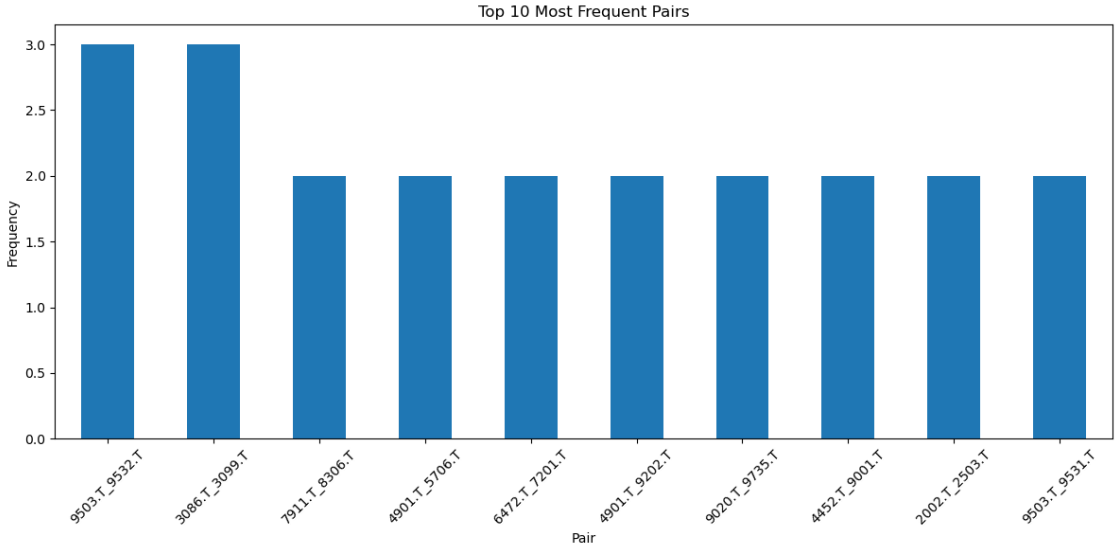

Cash Balance Over Time
$1,080,000
$1,060,000
$1,040,000
$1,020,000
$1,000,000
Cash Amount
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date


Cash as Percentage of Portfolio Value
Cash as % of Portfolio
100% Cash
102
101
100
99
98
Percentage (%)
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date


Number of Stocks in Portfolio Over Time
Number of Stocks
0
5
10
15
20
25
30
35
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date


Maximum Position Size as % of Portfolio Value
Percentage (%)
0.000
0.005
0.010
0.015
0.020
0.025
0.030
0.035
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
Date





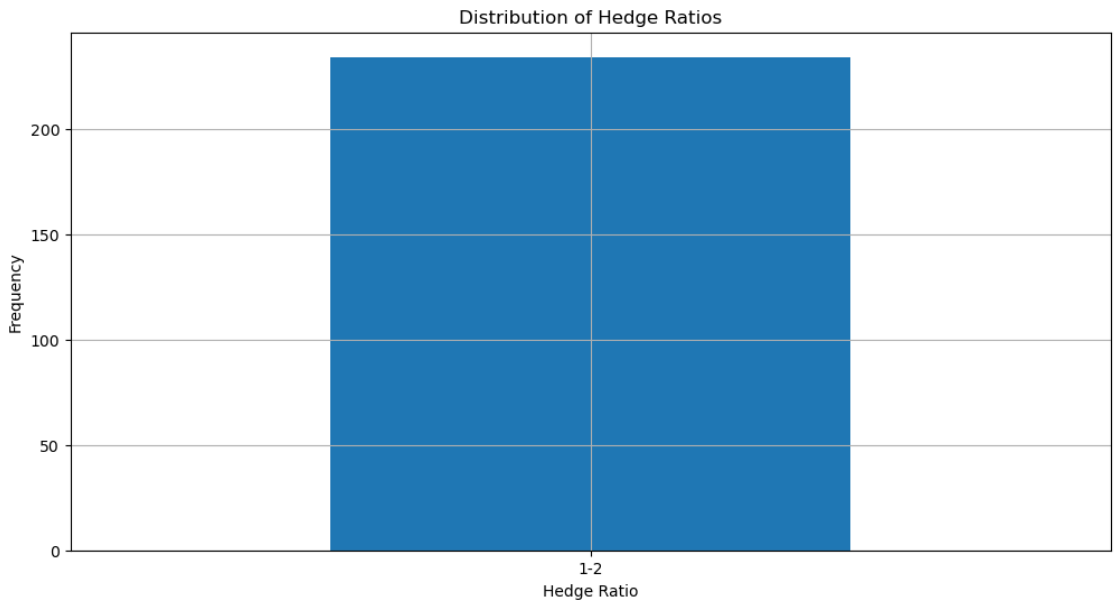

```
Top 10 Most Frequent Pairs:

              ticker1                        ticker1_name  \
pair
9503.T_9532.T  9503.T  THE.KANSAI.ELECTRIC.POWER.CO…INC.
3086.T_3099.T  3086.T          J.FRONT.RETAILING.CO…LTD.
7911.T_8306.T  7911.T                 TOPPAN.HOLDINGS.INC.
4901.T_5706.T  4901.T              FUJIFILM.HOLDINGS.CORP.
6472.T_7201.T  6472.T                          NTN.CORP.
4901.T_9202.T  4901.T              FUJIFILM.HOLDINGS.CORP.
```

9020.T_9735.T	9020.T	EAST.JAPAN.RAILWAY.CO.
4452.T_9001.T	4452.T	KAO.CORP.
2002.T_2503.T	2002.T	NISSHIN.SEIFUN.GROUP.INC.
9503.T_9531.T	9503.T	THE.KANSAI.ELECTRIC.POWER.CO…INC.

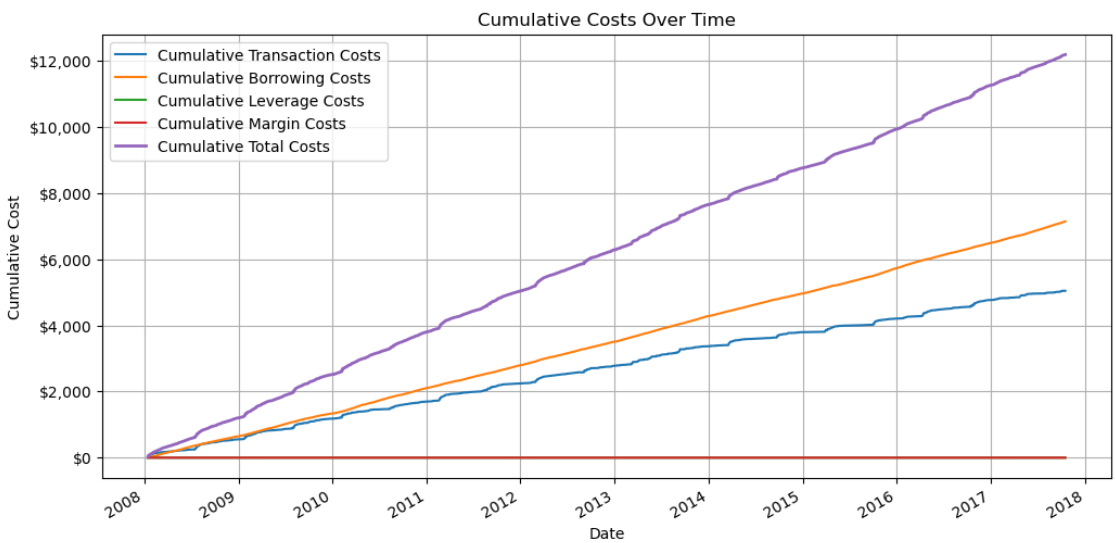
pair	ticker1_sector	ticker1_industry	ticker2 \
9503.T_9532.T	Transportation and Utilities	Electric Power	9532.T
3086.T_3099.T	Consumer Goods	Retail	3099.T
7911.T_8306.T	Capital Goods/Others	Other Manufacturing	8306.T
4901.T_5706.T	Materials	Chemicals	5706.T
6472.T_7201.T	Capital Goods/Others	Machinery	7201.T
4901.T_9202.T	Materials	Chemicals	9202.T
9020.T_9735.T	Transportation and Utilities	Railway & Bus	9735.T
4452.T_9001.T	Materials	Chemicals	9001.T
2002.T_2503.T	Consumer Goods	Foods	2503.T
9503.T_9531.T	Transportation and Utilities	Electric Power	9531.T

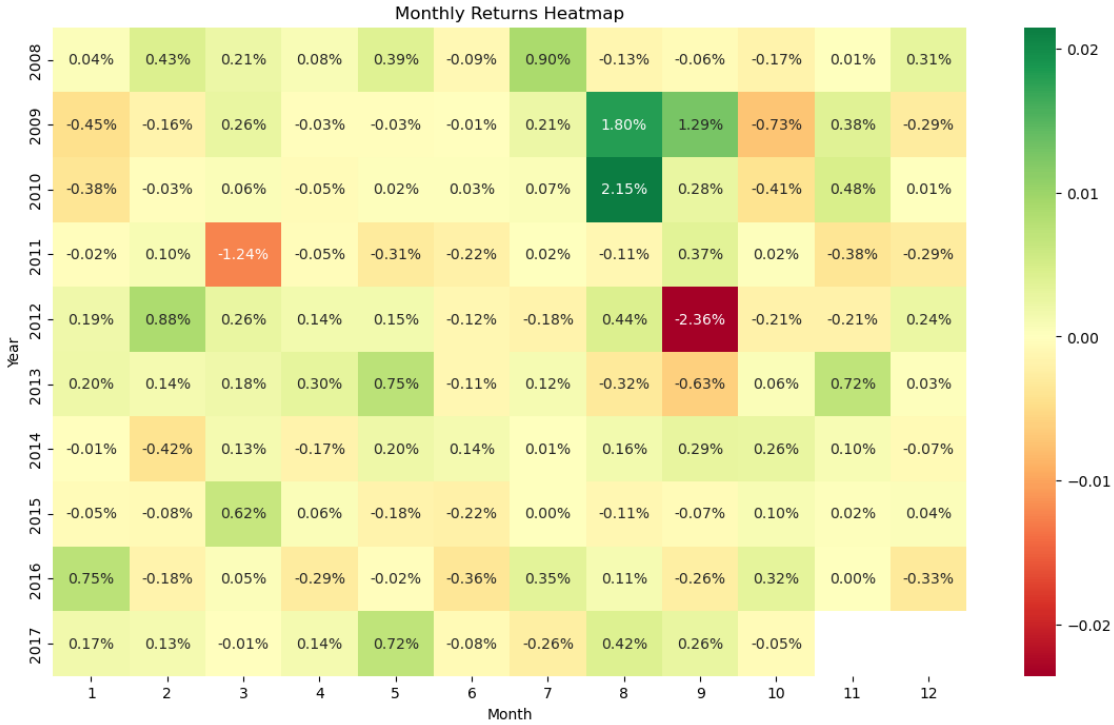
pair	ticker2_name \
9503.T_9532.T	OSAKA.GAS.CO…LTD.
3086.T_3099.T	ISETAN.MITSUKOSHI.HOLDINGS.LTD.
7911.T_8306.T	MITSUBISHI.UFJ.FINANCIAL.GROUP..INC.
4901.T_5706.T	MITSUI.MINING…SMELTING.CO.
6472.T_7201.T	NISSAN.MOTOR.CO…LTD.
4901.T_9202.T	ANA.HOLDINGS.INC.
9020.T_9735.T	SECOM.CO…LTD.
4452.T_9001.T	TOBU.RAILWAY.CO…LTD.
2002.T_2503.T	KIRIN.HOLDINGS.CO…LTD.
9503.T_9531.T	TOKYO.GAS.CO…LTD.

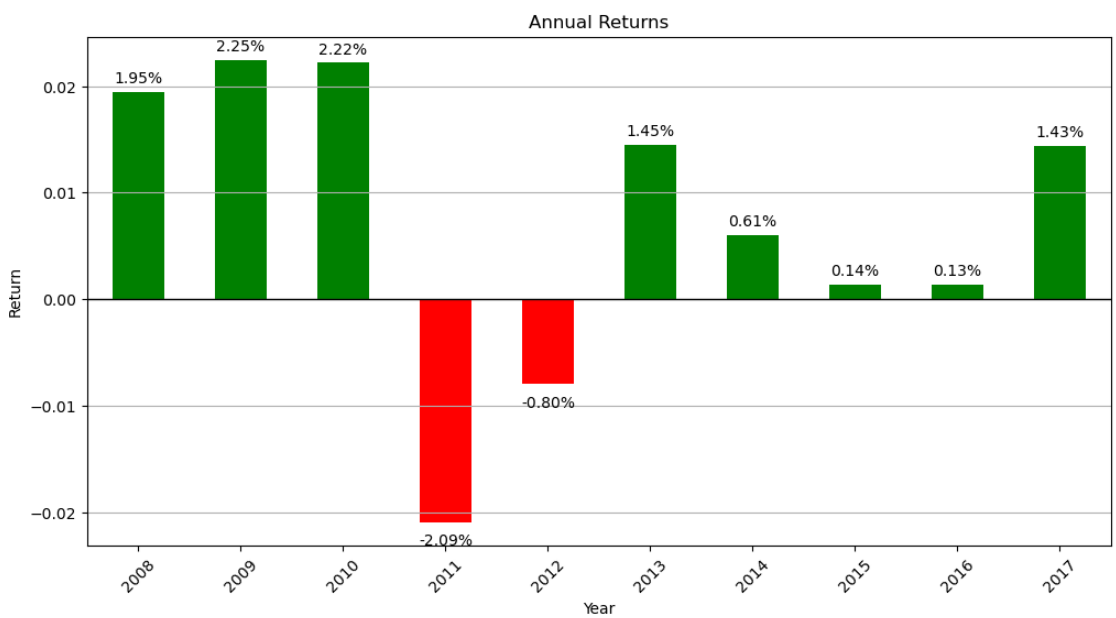
pair	ticker2_sector	ticker2_industry	count
9503.T_9532.T	Transportation and Utilities	Gas	3
3086.T_3099.T	Consumer Goods	Retail	3
7911.T_8306.T	Financials	Banking	2
4901.T_5706.T	Materials	Nonferrous Metals	2
6472.T_7201.T	Technology	Automobiles & Auto parts	2
4901.T_9202.T	Transportation and Utilities	Air Transport	2
9020.T_9735.T	Consumer Goods	Services	2
4452.T_9001.T	Transportation and Utilities	Railway & Bus	2
2002.T_2503.T	Consumer Goods	Foods	2
9503.T_9531.T	Transportation and Utilities	Gas	2




```
Pairs Trading Summary:
Total unique pairs: 218
Average hedge ratio: 1.00
```







	Metric	Value	Formatted Value
0	Net Total Return	0.0742	7.42%
1	Net Annualized Return	0.0076	0.76%
2	Net Volatility	0.0161	1.61%

3	Net Sharpe Ratio	0.4703	0.47
4	Gross Total Return	0.0868	8.68%
5	Gross Annualized Return	0.0088	0.88%
6	Gross Volatility	0.0161	1.61%
7	Gross Sharpe Ratio	0.5469	0.55
8	Net Max Drawdown	-0.0352	-3.52%
9	Net Calmar Ratio	0.2149	0.21
10	Gross Max Drawdown	-0.0332	-3.32%
11	Gross Calmar Ratio	0.2649	0.26
12	Net Win Rate	0.4908	49.08%
13	Gross Win Rate	0.5038	50.38%
14	Net Skewness	-1.8126	-1.81
15	Net Kurtosis	136.6496	136.65
16	Gross Skewness	-1.8071	-1.81
17	Gross Kurtosis	136.6164	136.62
18	Total Unique Pairs	218	218
19	Most Frequent Pair	9503.T_9532.T	9503.T_9532.T
20	Avg Hedge Ratio	1.0	1.00
21	Average Leverage	0.0461	4.61%
22	Max Leverage	0.051	5.10%
23	Min Leverage	0.0	0.00%
24	Total Gross Profit	86786.6049	$86,786.60
25	Total Net Profit	74227.5795	$74,227.58
26	Total Transaction Costs	5045.7913	$5,045.79
27	Avg Transaction Cost/Day	2.1077	$2.11
28	Total Transaction Cost Impact	0.0052	0.52%
29	Annual Transaction Cost Impact	0.0005	0.05%
30	Total Borrowing Costs	7142.936	$7,142.94
31	Avg Borrowing Cost/Day	2.9837	$2.98
32	Total Borrowing Cost Impact	0.0074	0.74%
33	Annual Borrowing Cost Impact	0.0007	0.07%
34	Total Leverage Costs	0	$0.00
35	Avg Leverage Cost/Day	0.0	$0.00
36	Total Margin Costs	0	$0.00
37	Avg Margin Cost/Day	0.0	$0.00
38	Total Costs	12188.7273	$12,188.73
39	Annual Txn+Borrow Cost Impact	0.0012	0.12%

Answer: We see that the months and years during the Great Financial Crisis the strategy does as good or better than the other months and years. This might indicate that our strategy is market netural. There are some years and months (especially August 2010) which is really bad. This might indicate that the covariance structure in the market changed a lot that month. However, this was most likely a shock as the subsequent months are not affected. Overall the strategy looks stable, but it does not earn or lose any money.

8 Question 8 (extra credit)

(extra credit: 1-5 points)

As a relatively new class, we're still developing some of the assignments, and like anyone, we're fallible. If you find a bug in the code, please report it to us, and we'll fix it. For doing so, we'll award you extra credit which scales linearly with the severity of the bug(s). Please describe your bug(s) below.

Your answer here - Wrong sign in plotting the spread in Question 5 and Question 6? - This is not really a big deal but it did take me some time to debug why my code failed. The problem is that some lines in the plotting in Question 5 and Question 6 removes the index and relies on iloc. This was caused by the fact that I did not provide any signal for the days in the estimation period which I think is reasonable. The line in my code that casued the issue was: `for idx, value in z_score.dropna().items():` which drops the nan values so the spread and signal is not of the same shape. The purpose of an index is to avoid these problems, so I have provided a fix below:

[ ]:
```
# In my code in the compute_signal I first had this for loop where I drop the␣
↪nan-values.
z_score = pd.DataFrame()
for idx, value in z_score.dropna().items():
    pass

pair_signals = signals[f"{ticker1}_{ticker2}"]
# entry_long = np.where(pair_signals['signal'] == -1)[0]
# entry_short = np.where(pair_signals['signal'] == 1)[0]
# exit_points = np.where(pair_signals['signal'] == 0)[0]
entry_long = pair_signals['signal'].loc[pair_signals['signal'] == -1]
entry_short = pair_signals['signal'].loc[pair_signals['signal'] == 1]
exit_points = pair_signals['signal'].loc[pair_signals['signal'] == 0]


# ax2.scatter(spread.index[entry_long], spread.iloc[entry_long],
#             color='green', marker='^', s=100, label='Long Position')
# ax2.scatter(spread.index[entry_short], spread.iloc[entry_short],
#             color='red', marker='v', s=100, label='Short Position')
# ax2.scatter(spread.index[exit_points], spread.iloc[exit_points],
#             color='black', marker='o', s=50, label='Flat Position')
# ax2.set_title(f'Spread and Signals: {ticker1} - {hr:.2f} * {ticker2}')

ax2.scatter(entry_long.index, spread.loc[entry_long.index],
            color='green', marker='^', s=100, label='Long Position')
ax2.scatter(entry_short.index, spread.loc[entry_short.index],
            color='red', marker='v', s=100, label='Short Position')
ax2.scatter(exit_points.index, spread.loc[exit_points.index],
            color='black', marker='o', s=50, label='Flat Position')
ax2.set_title(f'Spread and Signals: {ticker1} - {hr:.2f} * {ticker2}')
```

[ ]:
```
Text(0.5, 1.0, 'Spread and Signals: 9502.T - 1.00 * 9503.T')
```

9 Question 9 (ungraded)

(ungraded)

Here, feel free to experiment with different settings for the configuration dictionary or on the data and report any interesting results you discover. We won't grade this, but we will read it. Feel free to add any additional functionality to `backtest.py` to help you with your experiments. If you modify one of the functions pasted in a prior question, please leave the original code in that cell.

If you're still interested in exploring, here are several ideas for improvement:

1. To make this course more accessible, this backtester uses vanilla Python code (including Python loops), pandas dataframes, and Python objects. Even worse, it loops over Pandas dataframes, which is very inefficient. It would be a lot faster if we made more efficient choices, such as using NumPy arrays, taking advantage of vectorized functions, or using Cython. Try timing or profiling the `run_backtest` function and see where the bottlenecks are in the code. Then implement these more efficient methods for the one function where the most time is spent. How much does it speed up your code?
2. This backtester is written in a way which iterates over pairs formation periods and trading periods. This makes it easy to carve out functions for implementation, but it's not very realistic or extensible to other types of strategies. Moreover, it doesn't exactly correspond to the method of Gatev et al. 2006, which runs six separate portfolios, each of which has its own formation period and trading period, which are offset by one month. In this implementation, we only form new pairs every six months. Try changing the backtester to instead iterate over days, and form new pairs at the beginning or end of every month.
3. The `allocate_positions` function we propose is quite naive. Try to implement a smarter `allocate_positions` function which allocates capital more efficiently and controls risk a more carefully when there's a smaller number of pairs to trade. Similarly, you might want to increase risk when there's a larger number of pairs to trade–or not.
4. The return calculation is based on the entire amount of initial capital, even though under some position formation policies, such as ours, all of it may not be used at all times. Try implementing two return calculations: one which is based on committed capital, and another which is based on committed and allocated capital, as done in the Gatev et al. 2006 paper. If you're really ambitious, you could go even further and implement margin requirements, leverage limits, and margin calls. Then add a configuration option to enable or disable them to see how they affect the results.
5. There are some other robustness tests and metrics which we could add, from both the Gatev et al. 2006 paper and the broader literature. Try adding some of them. Comparing to a benchmark, computing more information about drawdowns, computing tables of the average, best, and worst returns, days, weeks, and months, and more would be welcome. Check out the quantstats package for ideas.